# COMP1832 PROGRAMMING FUNDAMENTALS FOR DATA SCIENCE

Tejesh Ramesh Bawa
001173097-8

# Python Portfolio Tasks

## 1. Introduction

The libraries used in this code is Matplotlib, NetworkX and the matplotlib.patches to display a portion of the public transportation network in the city of London via the graph data structure. The NetworkX is a strong library for creating and analysing complicated networks, resulting in an ideal choice for implementing transportation networks. It has a great capacity to integrate properties such as colour and positions with nodes and edges, resulting in a clear and customisable visual representation. Matplotlib supports the NetworkX by offering a flexible tool like the pyplot package for rendering nodes, edges and labels with appropriate colours and sizes, improving the overall clarity. The matplotlib.patches module is used to add geometric shapes like rectangle to emphasize the shared nodes, hence improving the graph's informational value. Overall, these libraries were chosen for their efficiency for being simple, flexible and robust in showing a visually appealing model of London's public transportation network.

## 2. Task Q1

In this task, a portion of 5 tubes from the London's public transportation network were used, which are the District, Piccadilly, Jubilee, Bakerloo and Northern Line. 6 stations were used from each line to exhibit the intricate relationship between the line which depicts a complex connected graph. The code below shows these 6 stations from each line represented as 6 nodes from each line (total of 30 nodes) which were connected to form the edges (total of 25 edges).

*Code*

```
MyGraph = nx.Graph()

# District Line nodes
MyGraph.add_node('S', npos=(-20, 30), ccn='#007d32')
MyGraph.add_node('A-D', npos=(0, 30), ccn='#007d32')
MyGraph.add_node('B-D', npos=(20, 30), ccn='#007d32')
MyGraph.add_node('C', npos=(40, 30), ccn='#007d32')
MyGraph.add_node('D-D', npos=(60, 30), ccn='#007d32')
MyGraph.add_node('E-D', npos=(76, 30), ccn='#007d32')

# Piccadilly Line nodes
MyGraph.add_node('P', npos=(0, 60), ccn='#0019a8')
MyGraph.add_node('A-P', npos=(0, 35), ccn='#0019a8')
MyGraph.add_node('B-P', npos=(20, 35), ccn='#0019a8')
MyGraph.add_node('F-P', npos=(37, 50), ccn='#0019a8')
MyGraph.add_node('G-P', npos=(60, 50), ccn='#0019a8')
MyGraph.add_node('H-P', npos=(76, 67), ccn='#0019a8')

# Jubilee Line nodes
MyGraph.add_node('I-J', npos=(35, 80), ccn='#a1a5a7')
MyGraph.add_node('F-J', npos=(41, 50), ccn='#a1a5a7')
MyGraph.add_node('D-J', npos=(60, 35), ccn='#a1a5a7')
MyGraph.add_node('J-J', npos=(76, 5), ccn='#a1a5a7')
MyGraph.add_node('K', npos=(95, 15), ccn='#a1a5a7')
MyGraph.add_node('Q', npos=(120, 20), ccn='#a1a5a7')

# Bakerloo Line nodes
MyGraph.add_node('R', npos=(20, 90), ccn='#b26300')
MyGraph.add_node('I-B', npos=(35, 85), ccn='#b26300')
MyGraph.add_node('L', npos=(55, 70), ccn='#b26300')
MyGraph.add_node('G-B', npos=(60, 55), ccn='#b26300')
```

```
MyGraph.add_node('M-B', npos=(76, 50), ccn='#b26300')
MyGraph.add_node('E-B', npos=(80, 32.5), ccn='#b26300')

# Northern Line nodes
MyGraph.add_node('O', npos=(80, 105), ccn='#000000')
MyGraph.add_node('N', npos=(80, 85), ccn='#000000')
MyGraph.add_node('H-N', npos=(80, 67), ccn='#000000')
MyGraph.add_node('M-N', npos=(80, 50), ccn='#000000')
MyGraph.add_node('E-N', npos=(80, 27.5), ccn='#000000')
MyGraph.add_node('J-N', npos=(80, 5), ccn='#000000')

# District Line edges
MyGraph.add_edge('S', 'A-D', cce='#007d32', distance='1.9 km')
MyGraph.add_edge('A-D', 'B-D', cce='#007d32', distance='1.8 km')
MyGraph.add_edge('B-D', 'C', cce='#007d32', distance='1.9 km')
MyGraph.add_edge('C', 'D-D', cce='#007d32', distance='2.8 km')
MyGraph.add_edge('D-D', 'E-D', cce='#007d32', distance='0.8 km')

# Piccadilly Line edges
MyGraph.add_edge('P', 'A-P', cce='#0019a8', distance='10.4 km')
MyGraph.add_edge('A-P', 'B-P', cce='#0019a8', distance='1.8 km')
MyGraph.add_edge('B-P', 'F-P', cce='#0019a8', distance='2.7 km')
MyGraph.add_edge('F-P', 'G-P', cce='#0019a8', distance='0.6 km')
MyGraph.add_edge('G-P', 'H-P', cce='#0019a8', distance='0.7 km')

# Jubilee Line edges
MyGraph.add_edge('I-J', 'F-J', cce='#a1a5a7', distance='2.4 km')
MyGraph.add_edge('F-J', 'D-J', cce='#a1a5a7', distance='2.1 km')
MyGraph.add_edge('D-J', 'J-J', cce='#a1a5a7', distance='1.1 km')
MyGraph.add_edge('J-J', 'K', cce='#a1a5a7', distance='12 km')
MyGraph.add_edge('K', 'Q', cce='#a1a5a7', distance='5.8 km')

# Bakerloo Line edges
MyGraph.add_edge('R', 'I-B', cce='#b26300', distance='0.5 km')
MyGraph.add_edge('I-B', 'L', cce='#b26300', distance='1.7 km')
MyGraph.add_edge('L', 'G-B', cce='#b26300', distance='0.9 km')
MyGraph.add_edge('G-B', 'M-B', cce='#b26300', distance='0.9 km')
MyGraph.add_edge('M-B', 'E-B', cce='#b26300', distance='1.3 km')

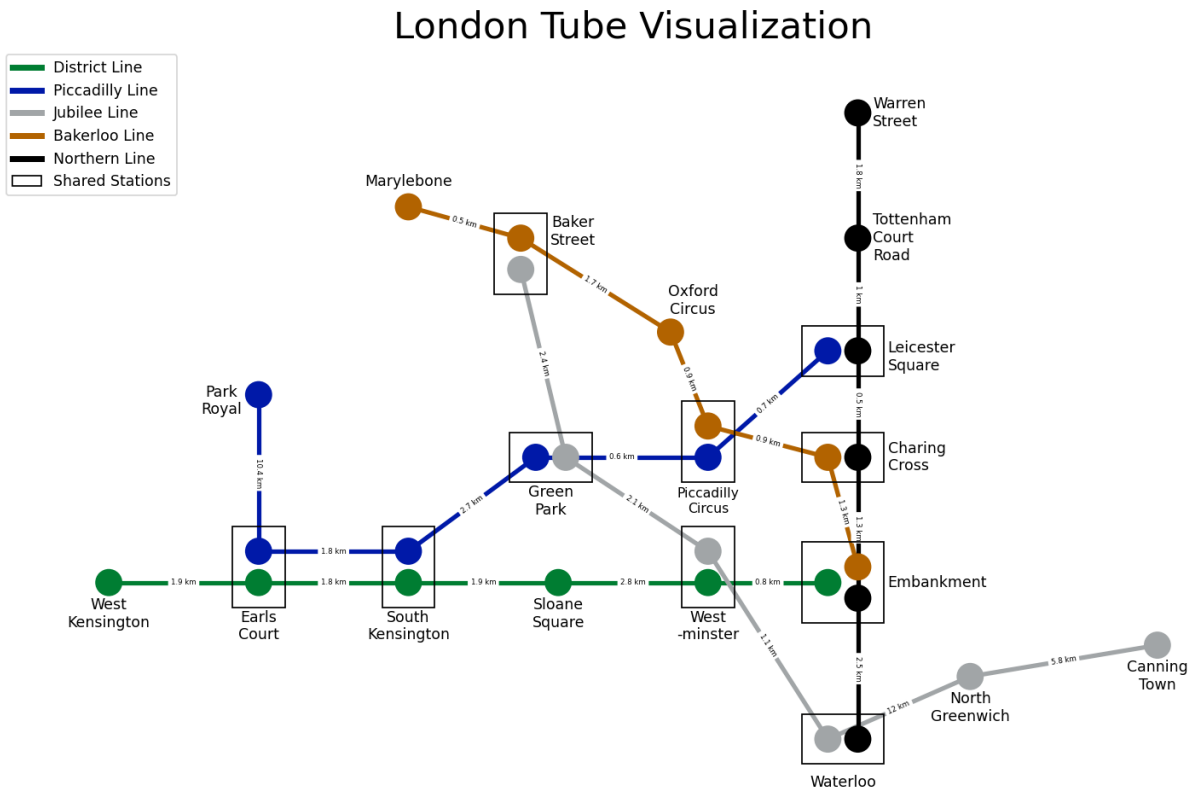
# Northern Line edges
MyGraph.add_edge('O', 'N', cce='#000000', distance='1.8 km')
MyGraph.add_edge('N', 'H-N', cce='#000000', distance='1 km')
MyGraph.add_edge('H-N', 'M-N', cce='#000000', distance='0.5 km')
MyGraph.add_edge('M-N', 'E-N', cce='#000000', distance='1.3 km')
MyGraph.add_edge('E-N', 'J-N', cce='#000000', distance='2.5 km')
```

*Code*

```
# Connect shared stations
MyGraph.add_edge('A-D', 'A-P', cce='none', weight=1)  # Shared connection
between District and Piccadilly
MyGraph.add_edge('B-D', 'B-P', cce='none', weight=1)  # Shared connection
between District and Piccadilly
MyGraph.add_edge('D-D', 'D-J', cce='none', weight=1)  # Shared connection
between District and Jubilee
MyGraph.add_edge('E-D', 'E-B', cce='none', weight=1)  # Shared connection
between District and Bakerloo
MyGraph.add_edge('E-D', 'E-N', cce='none', weight=1)  # Shared connection
between District and Northern
MyGraph.add_edge('E-B', 'E-N', cce='none', weight=1)  # Shared connection
```

```
between Bakerloo and Northern
MyGraph.add_edge('F-P', 'F-J', cce='none', weight=1)  # Shared connection
between Piccadilly and Jubilee
MyGraph.add_edge('G-P', 'G-B', cce='none', weight=1)  # Shared connection
between Piccadilly and Bakerloo
MyGraph.add_edge('H-P', 'H-N', cce='none', weight=1)  # Shared connection
between Piccadilly and Northern
MyGraph.add_edge('I-J', 'I-B', cce='none', weight=1)  # Shared connection
between Jubilee and Bakerloo
MyGraph.add_edge('J-J', 'J-N', cce='none', weight=1)  # Shared connection
between Jubilee and Northern
MyGraph.add_edge('M-B', 'M-N', cce='none', weight=1)  # Shared connection
between Bakerloo and Northern
```

*Output*




Furthermore, to verify whether the graph is connected or not, the use of nx.is_connected function was implemented and showed that the graph was connected.

## Code

```
is_connected = nx.is_connected(MyGraph)
print("Is the graph connected?", is_connected)
```

## Output

```
Is the graph connected? True
```

## 3. Task Q2

Each tube line and its station were assigned its own unique colours (same colours that are actually used by the TFL, for easier recognition) through the use of ccn attribute for the nodes and cce attributes for the edges.

### Code

For Nodes

```
MyGraph.add_node('S', npos=(-20, 30), ccn='#007d32')
```

For Edges

```
MyGraph.add_edge('S', 'A-D', cce='#007d32', distance='1.9 km')
```

For Rendering the Attributes

```
# Extract positions and colours
pos = nx.get_node_attributes(MyGraph, 'npos')
node_list = list(nx.get_node_attributes(MyGraph, 'ccn').values())
edge_list = list(nx.get_edge_attributes(MyGraph, 'cce').values())

# Draw nodes
nx.draw_networkx_nodes(MyGraph, pos, node_color=node_list, node_size=300,
ax=ax)

# Draw edges
nx.draw_networkx_edges(MyGraph, pos, edge_color=edge_list, width=3, ax=ax)
```

### Output




As many tube lines share the same station/nodes, for example, the District and Piccadilly Line both share the same station/node Earl's Court. Which in the code has been differentiated by depicting them as A-D and A-P, respectively. As the nodes have been coded separately, they have been connected together in the code below to ensure that the network is a connected graph.

### Code

```
# Connect shared stations
MyGraph.add_edge('A-D', 'A-P', cce='none', weight=1)  # Shared connection
between District and Piccadilly
MyGraph.add_edge('B-D', 'B-P', cce='none', weight=1)  # Shared connection
between District and Piccadilly
MyGraph.add_edge('D-D', 'D-J', cce='none', weight=1)  # Shared connection
between District and Jubilee
MyGraph.add_edge('E-D', 'E-B', cce='none', weight=1)  # Shared connection
between District and Bakerloo
MyGraph.add_edge('E-D', 'E-N', cce='none', weight=1)  # Shared connection
between District and Northern
MyGraph.add_edge('E-B', 'E-N', cce='none', weight=1)  # Shared connection
between Bakerloo and Northern
MyGraph.add_edge('F-P', 'F-J', cce='none', weight=1)  # Shared connection
between Piccadilly and Jubilee
MyGraph.add_edge('G-P', 'G-B', cce='none', weight=1)  # Shared connection
between Piccadilly and Bakerloo
MyGraph.add_edge('H-P', 'H-N', cce='none', weight=1)  # Shared connection
between Piccadilly and Northern
MyGraph.add_edge('I-J', 'I-B', cce='none', weight=1)  # Shared connection
between Jubilee and Bakerloo
```

```
MyGraph.add_edge('J-J', 'J-N', cce='none', weight=1)  # Shared connection
between Jubilee and Northern
MyGraph.add_edge('M-B', 'M-N', cce='none', weight=1)  # Shared connection
between Bakerloo and Northern
```

Visually the shared stations/nodes were lacking and hard to interpret whether the station were shared or not. To solve this problem, a rectangle was implemented which showed that the shared nodes into that rectangle which is showed below.

## Code

```
# Adding Rectangle to show Shared Nodes
Earls_Court = Rectangle((-3.5, 26), 7, 13, color='none', ec="black", lw=1)
ax.add_patch(Earls_Court)

South_Kensington = Rectangle((16.5, 26), 7, 13, color='none', ec="black",
lw=1)
ax.add_patch(South_Kensington)

Westminster = Rectangle((56.5, 26), 7, 13, color='none', ec="black", lw=1)
ax.add_patch(Westminster)

Piccadilly_Circus = Rectangle((56.5, 46), 7, 13, color='none', ec="black",
lw=1)
ax.add_patch(Piccadilly_Circus)

Baker_Street = Rectangle((31.5, 76), 7, 13, color='none', ec="black", lw=1)
ax.add_patch(Baker_Street)

Green_Park = Rectangle((33.5, 46), 11, 8, color='none', ec="black", lw=1)
ax.add_patch(Green_Park)

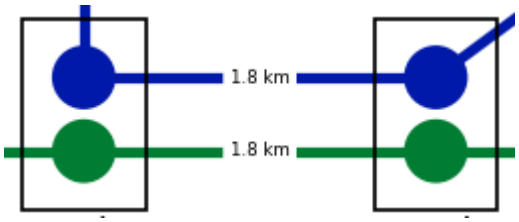
Leicester_Square = Rectangle((72.5, 63), 11, 8, color='none', ec="black",
lw=1)
ax.add_patch(Leicester_Square)

Charing_Cross = Rectangle((72.5, 46), 11, 8, color='none', ec="black",
lw=1)
ax.add_patch(Charing_Cross)

Waterloo = Rectangle((72.5, 1), 11, 8, color='none', ec="black", lw=1)
ax.add_patch(Waterloo)

Embankment = Rectangle((72.5, 23.5), 11, 13, color='none', ec="black",
lw=1)
ax.add_patch(Embankment)
```

## Output

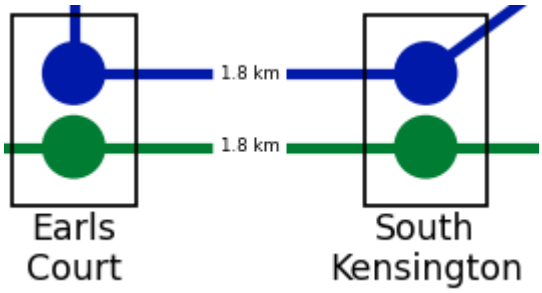



The stations are also labelled which is shown in the code below

Code

```
# Labelling the nodes
plt.text(-20, 25, s='West\nKensington', ha='center', va='center')
plt.text(-5, 59, s='Park\nRoyal', ha='center', va='center')
plt.text(0, 23, s='Earls\nCourt', ha='center', va='center')
plt.text(20, 23, s='South\nKensington', ha='center', va='center')
plt.text(40, 25, s='Sloane\nSquare', ha='center', va='center')
plt.text(60, 23, s='West\n-minster', ha='center', va='center')
plt.text(84, 30, s='Embankment', va='center')
plt.text(78, -2, s='Waterloo', ha='center', va='center')
plt.text(95, 10, s='North\nGreenwich', ha='center', va='center')
plt.text(39, 43, s='Green\nPark', ha='center', va='center')
plt.text(60, 43, s='Piccadilly\nCircus', ha='center', va='center', size=9)
plt.text(84, 50, s='Charing\nCross', va='center')
plt.text(84, 66, s='Leicester\nSquare', va='center')
plt.text(42, 86, s='Baker\nStreet', ha='center', va='center')
plt.text(58, 75, s='Oxford\nCircus', ha='center', va='center')
plt.text(82, 85, s='Tottenham\nCourt\nRoad', va='center')
plt.text(82, 105, s='Warren\nStreet', va='center')
plt.text(20, 94, s='Marylebone', ha='center', va='center')
plt.text(120, 15, s='Canning\nTown', ha='center', va='center')
```

Output

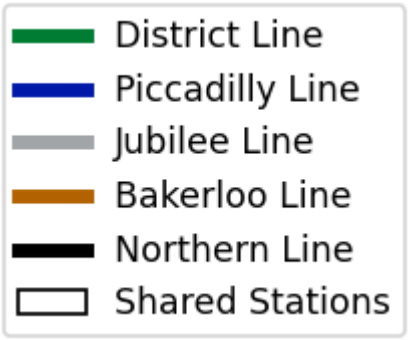



Furthermore, to improve the visualization for the commuters, a legend has been implemented to display the lines and their colour and is seen in the code below.

*Code*

```
# Plot the Legend
plt.legend(handles=[plt.Line2D([0], [0], color='#007d32', lw=4,
label='District Line'),
                    plt.Line2D([0], [0], color='#0019a8', lw=4,
label='Piccadilly Line'),
                    plt.Line2D([0], [0], color='#a1a5a7', lw=4,
label='Jubilee Line'),
                    plt.Line2D([0], [0], color='#b26300', lw=4,
label='Bakerloo Line'),
                    plt.Line2D([0], [0], color='#000000', lw=4,
label='Northern Line'),
                    Patch(facecolor="none", edgecolor="black",
label="Shared Stations"), ],
           loc='upper left')
```

*Output*




## 4. Task Q3

The distance attribute is used to show the distance between the neighbouring stations.

*Code*

```
MyGraph.add_edge('S', 'A-D', cce='#007d32', distance='1.9 km')
```

*Output*




## 5. Task Q4

One way to improve the visibility and usability of this map would be to improve the depiction of the shared stations and the distance. This is because, the commuters will get confused by the cluttered transfer stations (nodes) with their overlapping lines (edges) and the use of the rectangle. A better way would be, like how the TFL does by emphasize the boldness of the shared nodes outline

Another way is by adding time estimates instead of distance. This is because removing the distance will remove the clustered information that is not relevant to the commuters. For commuters, gauging time is far more important when travelling, which is not only improve its visibility but also its usability.

The Full Code and Output is appended to Appendix 1 (Python).

# R Portfolio Task

## 1. Task Q1 (Question 1)

The dataset was first imported using the readxl library using the read_xlsx() function. The dataset includes numerous fraud types and corresponding yearly statistics from 2012 to 2022. The aim was to develop independent data frames illustrating two distinct fraud categories: "Banking and Credit Industry Fraud" and "Consumer and Retail Fraud." The dplyr library helped make it easier to format the dataset.

```
# load the necessary libraries
library(readxl)
library(dplyr)

# loading the dataset
file_path <- "/content/cw_r.xlsx"
data <- read_xlsx(file_path, sheet = "Table 3d", skip=8)
```

The first data frame (Banking and Credit Industry Fraud) was created by slicing the rows 1-6 of the dataset. The select() function is executed to save the first 11 columns, which contain fraud categories and yearly fraud cases. To greater improve the readability, the first column is titled “Fraud Categories.” The data frame had 6 rows and 11 columns, with the first column being textual (chr) and the rest being numerical (num). The head() function exhibits the top rows, validating the contents inside the data frame, whereas str() offers the structural information regarding different data types and dimensions.

```
# create the first data frame: Banking and Credit Industry Fraud
dataframe_1 <- select(slice(data, 1:6), 1:11)
colnames(dataframe_1)[1] <- "Fraud Categories"
# Verify the data types
sapply(dataframe_1, class)
print(dataframe_1)
View(dataframe_1)

head(dataframe_1)
str(dataframe_1)
```

A tibble: 6 × 11

| Fraud Categories | Apr 2012 to Mar 2013 | Apr 2013 to Mar 2014 | Apr 2014 to Mar 2015 | Apr 2015 to Mar 2016 | Apr 2016 to Mar 2017 | Apr 2017 to Mar 2018 | Apr 2018 to Mar 2019 | Apr 2019 to Mar 2020 | Apr 2020 to Mar 2021 | Apr 2021 to Mar 2022 [note 28] |
|---|---|---|---|---|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| Banking and credit industry fraud | 306641 | 286234 | 320495 | 367714 | 380390 | 352064 | 386326 | 433336 | 384886 | 552024 |
| Cheque, plastic card and online bank accounts (not PSP) [note 5, 6] | 222272 | 226087 | 254479 | 293683 | 284607 | 266538 | 297234 | 338130 | 298815 | 475038 |
| Application fraud (excluding mortgages) | 80082 | 53390 | 58277 | 64166 | 84752 | 74027 | 78845 | 87033 | 80209 | 71475 |
| Mortgage related fraud | 2977 | 4337 | 3832 | 3544 | 3095 | 2708 | 2324 | 2085 | 1381 | 1297 |
| Mandate fraud | 1300 | 2288 | 3738 | 6193 | 7800 | 8618 | 7752 | 5929 | 4390 | 4084 |
| Dishonestly retaining a wrongful credit | 10 | 132 | 169 | 128 | 136 | 173 | 171 | 159 | 91 | 130 |

```
tibble [6 × 11] (S3: tbl_df/tbl/data.frame)
 $ Fraud Categories                  : chr [1:6] "Banking and credit industry fraud" "Cheque, plastic card and online bank accounts (not PSP) [note 5, 6]" '
 $ Apr 2012 to
Mar 2013            : num [1:6] 306641 222272 80082 2977 1300 ...
 $ Apr 2013 to
Mar 2014            : num [1:6] 286234 226087 53390 4337 2288 ...
 $ Apr 2014 to
Mar 2015            : num [1:6] 320495 254479 58277 3832 3738 ...
 $ Apr 2015 to
Mar 2016            : num [1:6] 367714 293683 64166 3544 6193 ...
 $ Apr 2016 to
Mar 2017            : num [1:6] 380390 284607 84752 3095 7800 ...
 $ Apr 2017 to
Mar 2018            : num [1:6] 352064 266538 74027 2708 8618 ...
 $ Apr 2018 to
Mar 2019            : num [1:6] 386326 297234 78845 2324 7752 ...
 $ Apr 2019 to
Mar 2020            : num [1:6] 433336 338130 87033 2085 5929 ...
 $ Apr 2020 to
Mar 2021            : num [1:6] 384886 298815 80209 1381 4390 ...
 $ Apr 2021 to
Mar 2022
[note 28]: num [1:6] 552024 475038 71475 1297 4084 ...
```

Similarly, rows from 31-38 of the datasets were sliced for the second data frame (Consumer and Retail Fraud). The same slicing and selecting functions were used to create the data frame, furthermore it has the same first column textual data (chr), and the remaining numerical data (num). The same head() and str() functions were used to check the contents and structure of the data frame.

```r
# create the second data frame: Consumer and retail fraud
dataframe_2 <- select(slice(data, 31:38), 1:11)
colnames(dataframe_2)[1] <- "Fraud Categories"
print(dataframe_2)
View(dataframe_2)

head(dataframe_2)
str(dataframe_2)
```

A tibble: 6 × 11

| Fraud Categories | Apr 2012 to Mar 2013 | Apr 2013 to Mar 2014 | Apr 2014 to Mar 2015 | Apr 2015 to Mar 2016 | Apr 2016 to Mar 2017 | Apr 2017 to Mar 2018 | Apr 2018 to Mar 2019 | Apr 2019 to Mar 2020 | Apr 2020 to Mar 2021 | Apr 2021 to Mar 2022 [note 28] |
|---|---|---|---|---|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| Consumer and retail fraud [note 14] | 47352 | 85560 | 98696 | 92442 | 107023 | 106249 | 110355 | 119716 | 155717 | 125560 |
| Online shopping and auctions | 22910 | 41625 | 42639 | 34836 | 36470 | 41422 | 51016 | 62509 | 97927 | 73937 |
| Consumer phone fraud | 557 | 1213 | 1151 | 1186 | 2293 | 923 | 2117 | 2769 | 4814 | 2595 |
| Door to door sales and bogus tradespeople | 2417 | 5671 | 4650 | 5168 | 5794 | 6695 | 5837 | 4784 | 4302 | 5367 |
| Other consumer and retail fraud [note 15] | 7723 | 15276 | 17109 | 11455 | 9279 | 9410 | 14808 | 20091 | 25725 | 24274 |
| Computer software service fraud | 8768 | 10784 | 22002 | 23538 | 31935 | 20804 | 21378 | 18919 | 18107 | 11864 |

```
tibble [8 × 11] (S3: tbl_df/tbl/data.frame)
 $ Fraud Categories                    : chr [1:8] "Consumer and retail fraud [note 14]" "Online shopping and auctions" "Consumer phone fraud"
 $ Apr 2012 to
Mar 2013            : num [1:8] 47352 22910 557 2417 7723 ...
 $ Apr 2013 to
Mar 2014            : num [1:8] 85560 41625 1213 5671 15276 ...
 $ Apr 2014 to
Mar 2015            : num [1:8] 98696 42639 1151 4650 17109 ...
 $ Apr 2015 to
Mar 2016            : num [1:8] 92442 34836 1186 5168 11455 ...
 $ Apr 2016 to
Mar 2017            : num [1:8] 107023 36470 2293 5794 9279 ...
 $ Apr 2017 to
Mar 2018            : num [1:8] 106249 41422 923 6695 9410 ...
 $ Apr 2018 to
Mar 2019            : num [1:8] 110355 51016 2117 5837 14808 ...
 $ Apr 2019 to
Mar 2020            : num [1:8] 119716 62509 2769 4784 20091 ...
 $ Apr 2020 to
Mar 2021            : num [1:8] 155717 97927 4814 4302 25725 ...
 $ Apr 2021 to
Mar 2022
[note 28]: num [1:8] 125560 73937 2595 5367 24274 ...
```

## 2. Task Q1 (Question 2)

In order to start, the data was transformed from a wide to a long format using the pivot_longer() function. This reorganization of data reduced the categories in one column (Fraud Categories) while also aligning with all the yearly data into another column (Year). Redesigning the dataset made it easier to visualise the patterns over time and compare fraud categories, allowing for more relevant conclusions.

Two line plots were used to analyse the central tendency of fraud cases, the first line plot was a normal one, while the second line plot was one with mean. To do that, the mean fraud cases for each year in both the data frames were determined using the group_by() and summerise() function. This demonstrated a consistent increase in the fraud cases over a period of time in both data frames. In the first data frame there is a high-volume of categories like Banking and credit industry fraud and Cheque, plastic card, and online bank accounts. Similarly, the second data frame also had an increase in mean fraud cases by categories of consumer and retail fraud and online shopping and auctions. The central tendency shows that there is a general increase in fraud cases over a period of time, with maximums noted at recent years which indicates that there is an increased risks and reporting of frauds.

The boxplots were used to visualise the spread of fraud cases using the jitter plots showing all individual data. In the first data frame, the large categorical quantities show significant variation over time. In contrast, all minor categories like mortgage-related fraud and dishonestly retaining wrongful credit show a more steady and stable trends. For the second data frame, categories like Consumer and retail fraud and Online shopping and auctions exhibiting the most unpredictability, while minor categories such as Consumer phone fraud and Ticket fraud maintain fairly steady.

These visualisations provided important insights regarding the central tendency and dispersion. Both data frames for the central tendency indicated that fraud cases increase over time. The boxplots highlighted that all the high-volume categories had significant fluctuation, whereas the smaller categories are steadier and more constant. Furthermore, both datasets have skewed distribution which shows that the bulk of the fraud cases fall into limited number of categories.

The code and output are shown in Appendix 2

## 3. Task Q1 (Question 3)

The data was first load using the read_xlsx() function, followed by slicing and selecting the dataset into two data frames for Banking and Credit Industry Fraud and Consumer and Retail Fraud. By creating two data frames it was easier to focus on each specific fraud categories to make detailed analysis. As the column names were confusing, they were subsequently changes to improve clarity and smoother data manipulation.

After the data frames were prepared, they were melted and reshaped, from a wider format to a longer format using the pivot_longer() function. This step pushed all the different years into one column ‘Years.’ This was an important step as it helps  by making it easy to analyse the trends over time while comparing it with the fraud categories.

The central tendency was analysed by calculating the mean fraud cases for each year using the group_by() and summarise() function. the line plot was chosen because it clearly depicts all the upward and downward trends, which helps to identify the categories with signification fraud activities against time.

The dispersion was analysed by boxplots by summarizing the spread, range and variability of the fraud cases as well as emphasizing any outliers. By adding jitter points it enhanced the box plot's interpretability to spot any anomalies and understand the distribution of the fraud cases.

## 4. Task Q2 (Question 1)

The table 5 in the dataset has been processed to analyse the offence and rate of offence per 1000 population in the region and county. This process is done using the dplyr library, by ensuring that all the important rows and columns were preserved and cleaned.

To start off, the column names were altered to improve the clarity to guarantee that the dataset is easier to further preprocess the data frame. To clean the Rate_per_1000_Population column, all the non-numeric data was removed using the gsub() function. All the rows that have missing values were eliminated to ensure that the dataset clean. Furthermore, the Area_Name, Number_of_Offences, Rate_per_1000_Population columns were chosen as required and the rest were dropped.

```
# loading the dataset
file_path <- "/content/cw_r.xlsx"
data <- read_xlsx(file_path, sheet = "Table 5", skip=9)

# renaming the columns for clarity
colnames(data) <- c("Area_Code", "Area_Name", "Number_of_Offences",
                    "Rate_per_1000_Population",
"Change_from_Previous_Year")

# remove non-numeric rows from the Rate_per_1000_Population column
data <- data %>%
  mutate(Rate_per_1000_Population = as.numeric(gsub("[^0-9.]", "",
Rate_per_1000_Population)))

# filtering and cleaning the data
clean_df <- data %>%
  filter(!is.na(Area_Name)) %>% # removing rows with any missing
Area_Name
  filter(!is.na(Number_of_Offences) & !is.na(Rate_per_1000_Population))
%>% # removing rows with any missing data
  select(Area_Name, Number_of_Offences, Rate_per_1000_Population) #
Keep all relevant columns

# converting columns to numeric
clean_df <- clean_df %>%
  mutate(
    Number_of_Offences = as.numeric(Number_of_Offences),
    Rate_per_1000_Population = as.numeric(Rate_per_1000_Population)
  ) %>%
  drop_na() # Rremove all rows with any invalid numeric values

print(clean_df)
```

The data frame was further pre-processed to assign the regions to the counties using the case_when() function. All the region rows were identified, and they were passed with their respective counties using the fill() function, this helps with providing a clear link between regions and counties. The "ENGLAND AND WALES" and "ENGLAND" rows were dropped because they indicated that they were for countries. Finally, there were 4 columns named Region, County, Number_of_Offences, and Rate_per_1000_Population, which will be used for further analysis.

```
# preprocess the dataset
processed_data <- clean_df %>%
  mutate(
    Region = case_when(
      Area_Name %in% c("North East", "North West", "Yorkshire and The
Humber",
                       "East Midlands", "West Midlands", "East",
                       "South East", "South West", "WALES") ~
Area_Name,
      TRUE ~ NA_character_
    )
  ) %>%
  fill(Region, .direction = "down") %>% # Assign all regions to
counties
  filter(!Area_Name %in% c("ENGLAND AND WALES [note 13]", "ENGLAND",
"London [note 14]")) %>% # Excluding the first 2 rows
  rename(County = Area_Name) %>% # renaming Area_Name to County
  select(Region, County, Number_of_Offences, Rate_per_1000_Population)
# selecting relevant columns

print(processed_data)
```

```
# A tibble: 52 × 4
   Region     County            Number_of_Offences Rate_per_1000_Population
   <chr>      <chr>                          <dbl>                    <dbl>
 1 North East North East                  10718                     4.00
 2 North East Cleveland                    2258                     3.96
 3 North East Durham                       2606                     4.07
 4 North East Northumbria                  5854                     3.98
 5 North West North West                  36433                     4.95
 6 North West Cheshire                     6019                     5.63
 7 North West Cumbria                      2157                     4.32
 8 North West Greater Manchester          14335                     5.03
 9 North West Lancashire                   7195                     4.75
10 North West Merseyside                   6727                     4.69
# i 42 more rows
```

## 5. Task Q2 (Question 2)

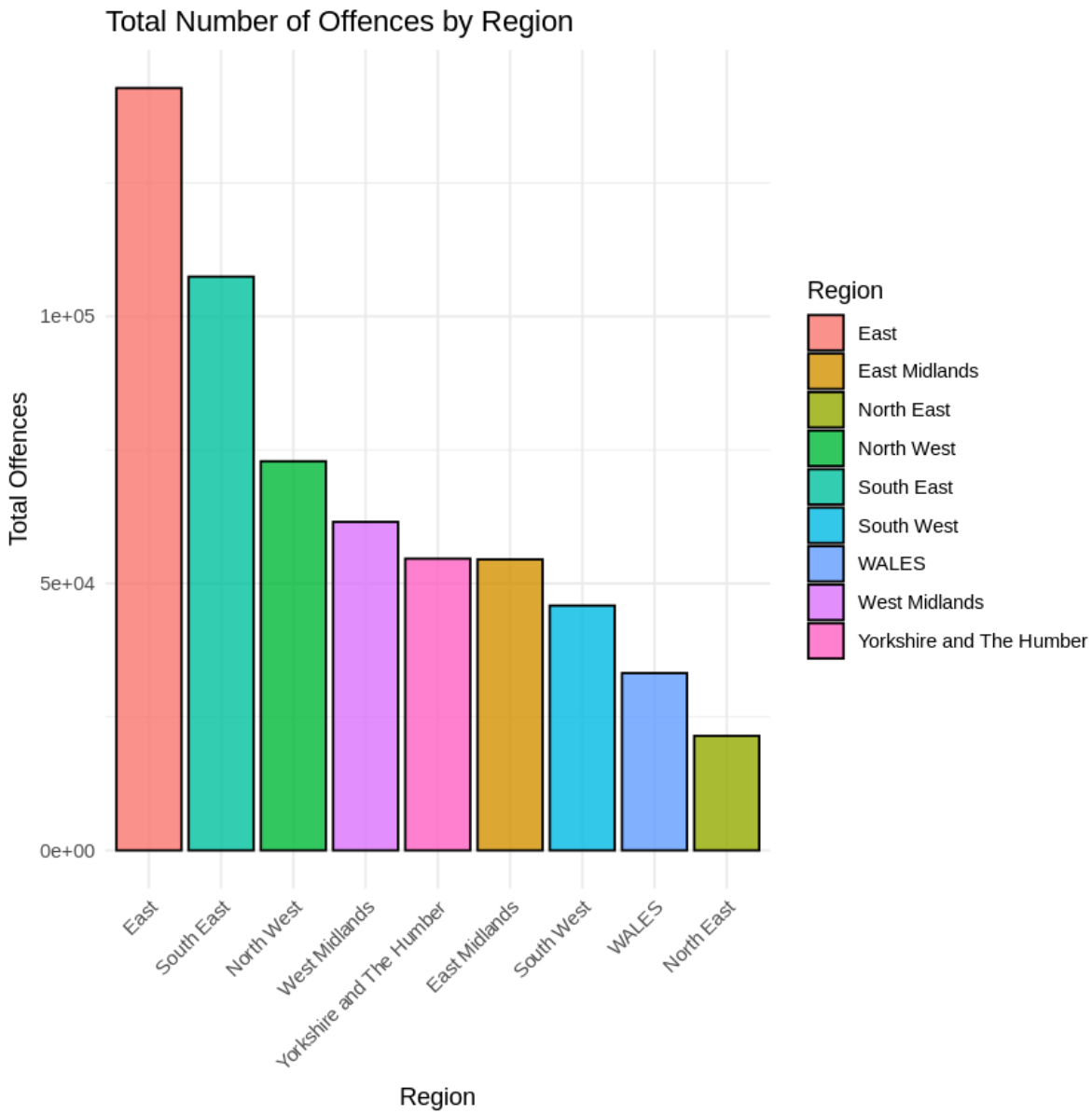
The code uses the dplyr to examine the trends in area against both number of offences and rete per 1000 population by aggregating the 2 sets. These parameters are set to visualise as graphs to easily compare the locations.

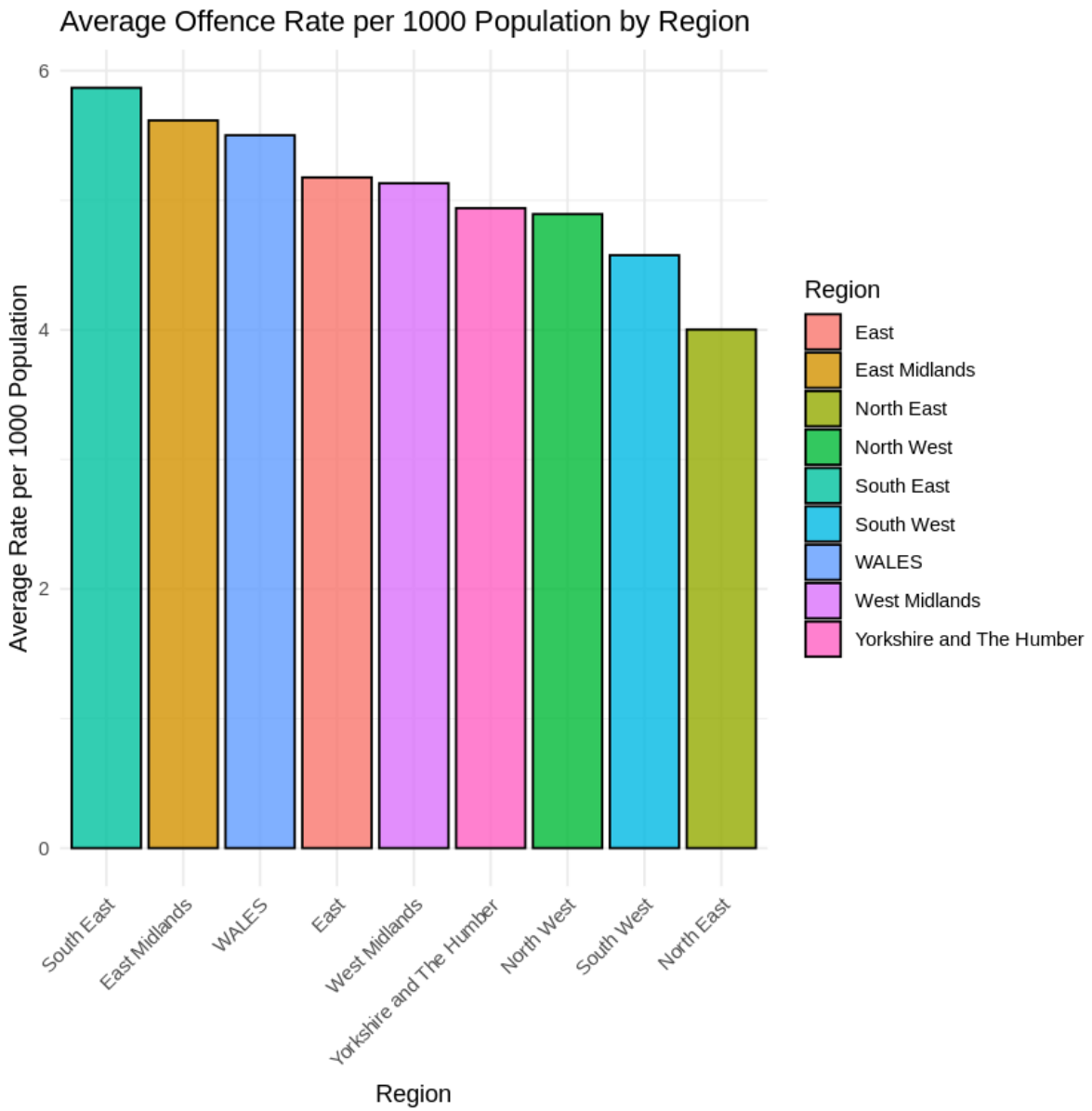
```
# summarize the data at the region level
```

```r
region_summary <- processed_data %>%
  group_by(Region) %>%
  summarize(
    Total_Offences = sum(Number_of_Offences, na.rm = TRUE),
    Avg_Rate_per_1000 = mean(Rate_per_1000_Population, na.rm = TRUE)
  )

# Bar plot for Total Offences by Region
ggplot(region_summary, aes(x = reorder(Region, -Total_Offences), y =
Total_Offences, fill = Region)) +
  geom_bar(stat = "identity", alpha = 0.8, color = "black") +
  theme_minimal() +
  labs(
    title = "Total Number of Offences by Region",
    x = "Region",
    y = "Total Offences"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

# Bar plot for Average Offence Rate per 1000 Population by Region
ggplot(region_summary, aes(x = reorder(Region, -Avg_Rate_per_1000), y =
Avg_Rate_per_1000, fill = Region)) +
  geom_bar(stat = "identity", alpha = 0.8, color = "black") +
  theme_minimal() +
  labs(
    title = "Average Offence Rate per 1000 Population by Region",
    x = "Region",
    y = "Average Rate per 1000 Population"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```


Total Number of Offences by Region
Total Offences
1e+05
5e+04
0e+00
East
South East
North West
West Midlands
Yorkshire and The Humber
East Midlands
South West
WALES
North East
Region
Region
East
East Midlands
North East
North West
South East
South West
WALES
West Midlands
Yorkshire and The Humber




The first bar plot shows the overall number of offences per region. The figure shows that the East region has the most total offences, followed by the South East and North West. The second bar plot show the rate of offences per 1000 population per region. The figure shows that the South East region has the highest rate, followed by East Midlands and Wales.

Bar plots are well-suited for this study as they are easy to compare all the quantitative variables across the regions with visual ranking. By ordering the bars in descending orders, it highlights the regions with the most important metrics are clearly evident. Furthermore, separating the number of offences and rate of offences into 2 separate plots allows for independent examination which also reduces the cluttered information.

With this it is clearly noted that the South East region has the greatest number of offences while also having the highest rate of offences per 1000 population.

## 6. Task Q2 (Question 3)

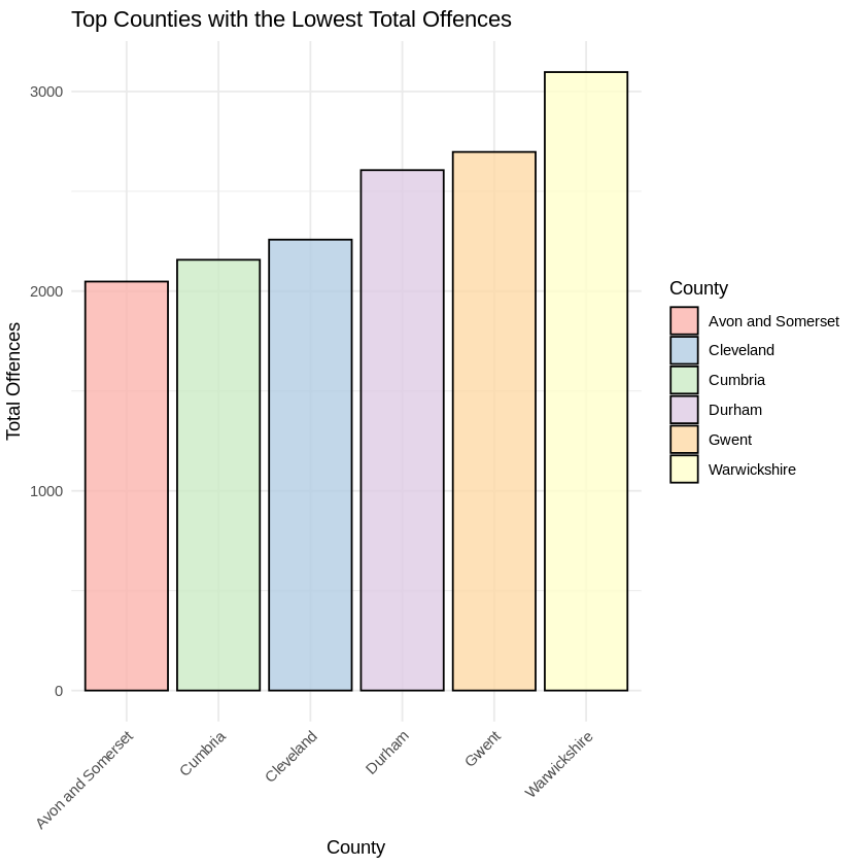
The bar plot highlights the counties with lowest total offences and ranking them in an ascending order. The three lowest total offences are Avon and Somerset, Cumbria and Cleveland.

```
# summarize offences by county
county_summary <- processed_data %>%
  group_by(County) %>%
    summarize(Total_Offences = sum(Number_of_Offences, na.rm = TRUE)) %>%
    arrange(Total_Offences) # Sort by total offences in ascending order

  # selecting the bottom counties
  bottom_counties <- head(county_summary)

  # visualize the bottom counties using a bar plot
  ggplot(bottom_counties, aes(x = reorder(County, Total_Offences), y = Total_Offences, fill = County)) +
    geom_bar(stat = "identity", color = "black", alpha = 0.8) +
    theme_minimal() +
    labs(
      title = "Top Counties with the Lowest Total Offences",
      x = "County",
      y = "Total Offences"
    ) +
    theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
    scale_fill_brewer(palette = "Pastel1") # Optional color palette
```

The bar chart plot was used as it is easier to visualise and compare the offences with each county while also ranking them in ascending order. Due to this approach, it provides an intuitive and straightforward representation for identifying the lowest counts of offence. It is also easy to read as it does not have cluttered data which will need to be further refined.


Top Counties with the Lowest Total Offences
Total Offences
3000
2000
1000
0
Avon and Somerset
Cumbria
Cleveland
Durham
Gwent
Warwickshire
County
County
Avon and Somerset
Cleveland
Cumbria
Durham
Gwent
Warwickshire

# Appendix 1 (Python)

## Code

```
import matplotlib.pyplot as plt
import networkx as nx
from matplotlib.patches import Rectangle, Patch

MyGraph = nx.Graph()

# District Line nodes
MyGraph.add_node('S', npos=(-20, 30), ccn='#007d32')
MyGraph.add_node('A-D', npos=(0, 30), ccn='#007d32')
MyGraph.add_node('B-D', npos=(20, 30), ccn='#007d32')
MyGraph.add_node('C', npos=(40, 30), ccn='#007d32')
MyGraph.add_node('D-D', npos=(60, 30), ccn='#007d32')
MyGraph.add_node('E-D', npos=(76, 30), ccn='#007d32')

# Piccadilly Line nodes
MyGraph.add_node('P', npos=(0, 60), ccn='#0019a8')
MyGraph.add_node('A-P', npos=(0, 35), ccn='#0019a8')
MyGraph.add_node('B-P', npos=(20, 35), ccn='#0019a8')
MyGraph.add_node('F-P', npos=(37, 50), ccn='#0019a8')
MyGraph.add_node('G-P', npos=(60, 50), ccn='#0019a8')
MyGraph.add_node('H-P', npos=(76, 67), ccn='#0019a8')

# Jubilee Line nodes
MyGraph.add_node('I-J', npos=(35, 80), ccn='#a1a5a7')
MyGraph.add_node('F-J', npos=(41, 50), ccn='#a1a5a7')
MyGraph.add_node('D-J', npos=(60, 35), ccn='#a1a5a7')
MyGraph.add_node('J-J', npos=(76, 5), ccn='#a1a5a7')
MyGraph.add_node('K', npos=(95, 15), ccn='#a1a5a7')
MyGraph.add_node('Q', npos=(120, 20), ccn='#a1a5a7')

# Bakerloo Line nodes
MyGraph.add_node('R', npos=(20, 90), ccn='#b26300')
MyGraph.add_node('I-B', npos=(35, 85), ccn='#b26300')
MyGraph.add_node('L', npos=(55, 70), ccn='#b26300')
MyGraph.add_node('G-B', npos=(60, 55), ccn='#b26300')
MyGraph.add_node('M-B', npos=(76, 50), ccn='#b26300')
MyGraph.add_node('E-B', npos=(80, 32.5), ccn='#b26300')

# Northern Line nodes
MyGraph.add_node('O', npos=(80, 105), ccn='#000000')
MyGraph.add_node('N', npos=(80, 85), ccn='#000000')
MyGraph.add_node('H-N', npos=(80, 67), ccn='#000000')
MyGraph.add_node('M-N', npos=(80, 50), ccn='#000000')
MyGraph.add_node('E-N', npos=(80, 27.5), ccn='#000000')
MyGraph.add_node('J-N', npos=(80, 5), ccn='#000000')

# District Line edges
MyGraph.add_edge('S', 'A-D', cce='#007d32', distance='1.9 km')
MyGraph.add_edge('A-D', 'B-D', cce='#007d32', distance='1.8 km')
MyGraph.add_edge('B-D', 'C', cce='#007d32', distance='1.9 km')
MyGraph.add_edge('C', 'D-D', cce='#007d32', distance='2.8 km')
MyGraph.add_edge('D-D', 'E-D', cce='#007d32', distance='0.8 km')

# Piccadilly Line edges
MyGraph.add_edge('P', 'A-P', cce='#0019a8', distance='10.4 km')
MyGraph.add_edge('A-P', 'B-P', cce='#0019a8', distance='1.8 km')
MyGraph.add_edge('B-P', 'F-P', cce='#0019a8', distance='2.7 km')
```

```
MyGraph.add_edge('F-P', 'G-P', cce='#0019a8', distance='0.6 km')
MyGraph.add_edge('G-P', 'H-P', cce='#0019a8', distance='0.7 km')

# Jubilee Line edges
MyGraph.add_edge('I-J', 'F-J', cce='#a1a5a7', distance='2.4 km')
MyGraph.add_edge('F-J', 'D-J', cce='#a1a5a7', distance='2.1 km')
MyGraph.add_edge('D-J', 'J-J', cce='#a1a5a7', distance='1.1 km')
MyGraph.add_edge('J-J', 'K', cce='#a1a5a7', distance='12 km')
MyGraph.add_edge('K', 'Q', cce='#a1a5a7', distance='5.8 km')

# Bakerloo Line edges
MyGraph.add_edge('R', 'I-B', cce='#b26300', distance='0.5 km')
MyGraph.add_edge('I-B', 'L', cce='#b26300', distance='1.7 km')
MyGraph.add_edge('L', 'G-B', cce='#b26300', distance='0.9 km')
MyGraph.add_edge('G-B', 'M-B', cce='#b26300', distance='0.9 km')
MyGraph.add_edge('M-B', 'E-B', cce='#b26300', distance='1.3 km')

# Northern Line edges
MyGraph.add_edge('O', 'N', cce='#000000', distance='1.8 km')
MyGraph.add_edge('N', 'H-N', cce='#000000', distance='1 km')
MyGraph.add_edge('H-N', 'M-N', cce='#000000', distance='0.5 km')
MyGraph.add_edge('M-N', 'E-N', cce='#000000', distance='1.3 km')
MyGraph.add_edge('E-N', 'J-N', cce='#000000', distance='2.5 km')

# Connect shared stations
MyGraph.add_edge('A-D', 'A-P', cce='none', weight=1)  # Shared connection
between District and Piccadilly
MyGraph.add_edge('B-D', 'B-P', cce='none', weight=1)  # Shared connection
between District and Piccadilly
MyGraph.add_edge('D-D', 'D-J', cce='none', weight=1)  # Shared connection
between District and Jubilee
MyGraph.add_edge('E-D', 'E-B', cce='none', weight=1)  # Shared connection
between District and Bakerloo
MyGraph.add_edge('E-D', 'E-N', cce='none', weight=1)  # Shared connection
between District and Northern
MyGraph.add_edge('E-B', 'E-N', cce='none', weight=1)  # Shared connection
between Bakerloo and Northern
MyGraph.add_edge('F-P', 'F-J', cce='none', weight=1)  # Shared connection
between Piccadilly and Jubilee
MyGraph.add_edge('G-P', 'G-B', cce='none', weight=1)  # Shared connection
between Piccadilly and Bakerloo
MyGraph.add_edge('H-P', 'H-N', cce='none', weight=1)  # Shared connection
between Piccadilly and Northern
MyGraph.add_edge('I-J', 'I-B', cce='none', weight=1)  # Shared connection
between Jubilee and Bakerloo
MyGraph.add_edge('J-J', 'J-N', cce='none', weight=1)  # Shared connection
between Jubilee and Northern
MyGraph.add_edge('M-B', 'M-N', cce='none', weight=1)  # Shared connection
between Bakerloo and Northern

# Create a plot
fig, ax = plt.subplots()

# Extract positions and colours
pos = nx.get_node_attributes(MyGraph, 'npos')
node_list = list(nx.get_node_attributes(MyGraph, 'ccn').values())
edge_list = list(nx.get_edge_attributes(MyGraph, 'cce').values())

# Draw nodes
nx.draw_networkx_nodes(MyGraph, pos, node_color=node_list, node_size=300,
ax=ax)
```

```
# Draw edges
nx.draw_networkx_edges(MyGraph, pos, edge_color=edge_list, width=3, ax=ax)

# Get distances for edge labels
edge_labels = nx.get_edge_attributes(MyGraph, 'distance')
nx.draw_networkx_edge_labels(MyGraph, pos, edge_labels=edge_labels,
font_size=5, ax=ax)

# Write the Title
plt.title("London Tube Visualization", size=25)

# Plot the Legend
plt.legend(handles=[plt.Line2D([0], [0], color='#007d32', lw=4,
label='District Line'),
                    plt.Line2D([0], [0], color='#0019a8', lw=4,
label='Piccadilly Line'),
                    plt.Line2D([0], [0], color='#a1a5a7', lw=4,
label='Jubilee Line'),
                    plt.Line2D([0], [0], color='#b26300', lw=4,
label='Bakerloo Line'),
                    plt.Line2D([0], [0], color='#000000', lw=4,
label='Northern Line'),
                    Patch(facecolor="none", edgecolor="black",
label="Shared Stations"), ],
           loc='upper left')

# Adding Rectangle to show Shared Nodes
Earls_Court = Rectangle((-3.5, 26), 7, 13, color='none', ec="black", lw=1)
ax.add_patch(Earls_Court)

South_Kensington = Rectangle((16.5, 26), 7, 13, color='none', ec="black",
lw=1)
ax.add_patch(South_Kensington)

Westminster = Rectangle((56.5, 26), 7, 13, color='none', ec="black", lw=1)
ax.add_patch(Westminster)

Piccadilly_Circus = Rectangle((56.5, 46), 7, 13, color='none', ec="black",
lw=1)
ax.add_patch(Piccadilly_Circus)

Baker_Street = Rectangle((31.5, 76), 7, 13, color='none', ec="black", lw=1)
ax.add_patch(Baker_Street)

Green_Park = Rectangle((33.5, 46), 11, 8, color='none', ec="black", lw=1)
ax.add_patch(Green_Park)

Leicester_Square = Rectangle((72.5, 63), 11, 8, color='none', ec="black",
lw=1)
ax.add_patch(Leicester_Square)

Charing_Cross = Rectangle((72.5, 46), 11, 8, color='none', ec="black",
lw=1)
ax.add_patch(Charing_Cross)

Waterloo = Rectangle((72.5, 1), 11, 8, color='none', ec="black", lw=1)
ax.add_patch(Waterloo)

Embankment = Rectangle((72.5, 23.5), 11, 13, color='none', ec="black",
lw=1)
```

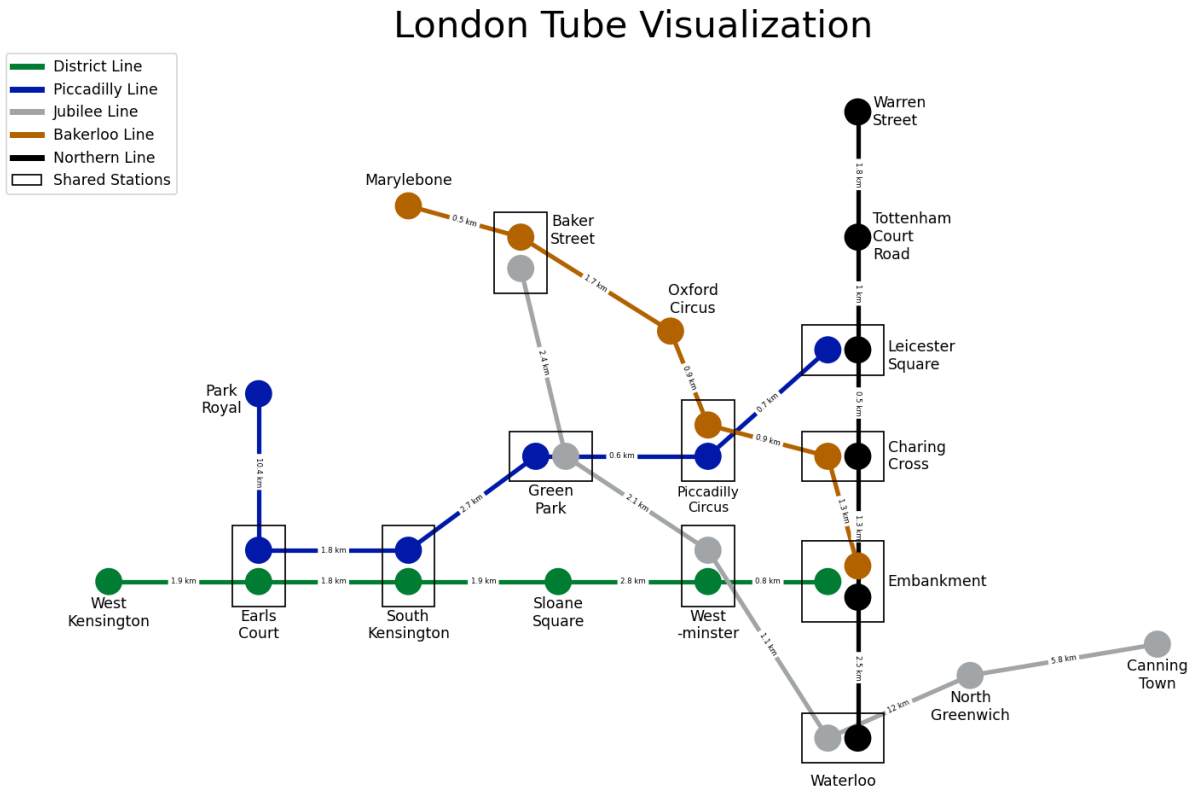
```
ax.add_patch(Embankment)


# Labelling the nodes
plt.text(-20, 25, s='West\nKensington', ha='center', va='center')
plt.text(-5, 59, s='Park\nRoyal', ha='center', va='center')
plt.text(0, 23, s='Earls\nCourt', ha='center', va='center')
plt.text(20, 23, s='South\nKensington', ha='center', va='center')
plt.text(40, 25, s='Sloane\nSquare', ha='center', va='center')
plt.text(60, 23, s='West\n-minster', ha='center', va='center')
plt.text(84, 30, s='Embankment', va='center')
plt.text(78, -2, s='Waterloo', ha='center', va='center')
plt.text(95, 10, s='North\nGreenwich', ha='center', va='center')
plt.text(39, 43, s='Green\nPark', ha='center', va='center')
plt.text(60, 43, s='Piccadilly\nCircus', ha='center', va='center', size=9)
plt.text(84, 50, s='Charing\nCross', va='center')
plt.text(84, 66, s='Leicester\nSquare', va='center')
plt.text(42, 86, s='Baker\nStreet', ha='center', va='center')
plt.text(58, 75, s='Oxford\nCircus', ha='center', va='center')
plt.text(82, 85, s='Tottenham\nCourt\nRoad', va='center')
plt.text(82, 105, s='Warren\nStreet', va='center')
plt.text(20, 94, s='Marylebone', ha='center', va='center')
plt.text(120, 15, s='Canning\nTown', ha='center', va='center')

plt.axis("off")
plt.show()

is_connected = nx.is_connected(MyGraph)
print("Is the graph connected?", is_connected)
```

## Output



# Appendix 2 (R)

## Code (Date Frame 1 Task 2) And Output

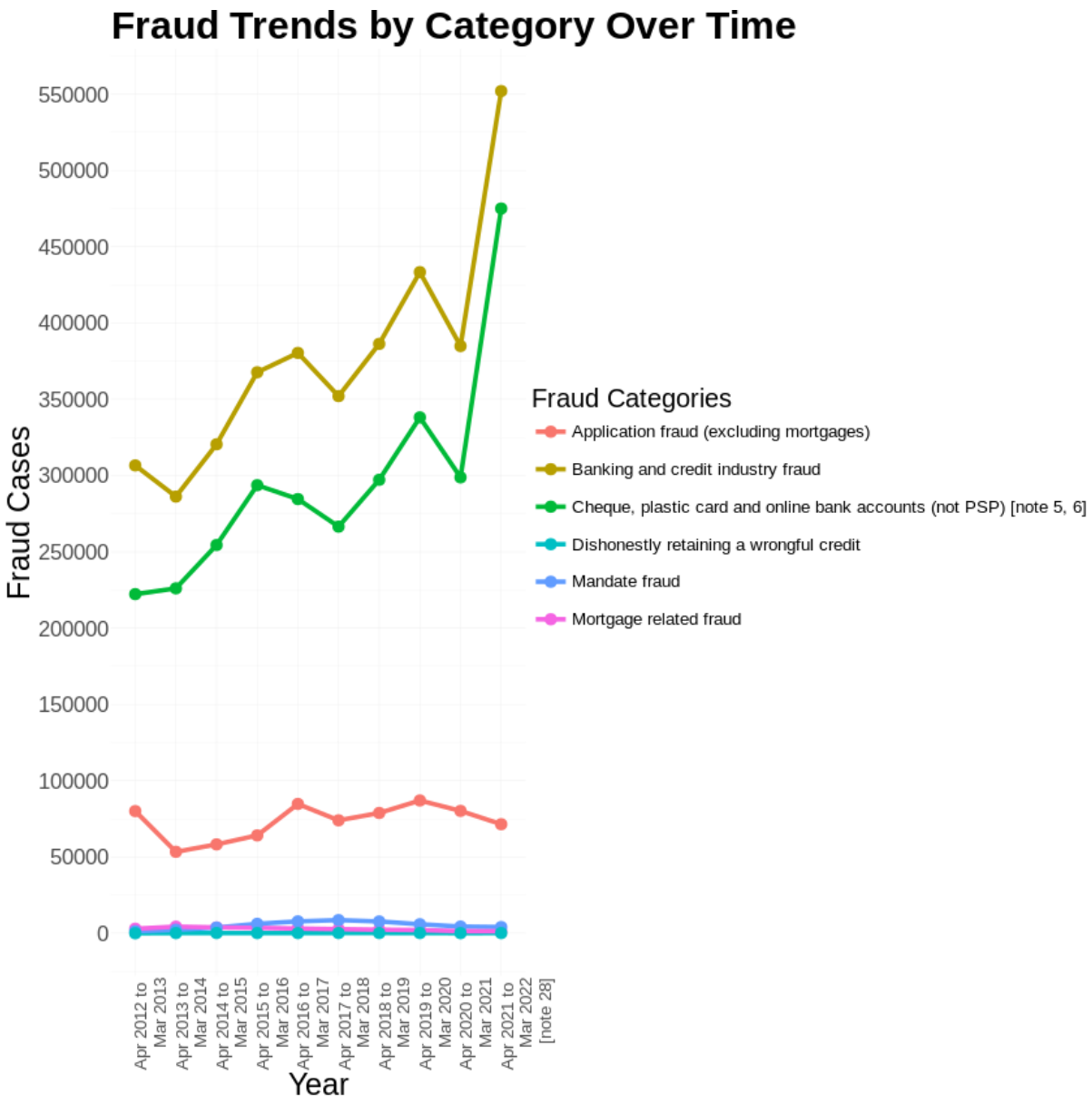
```
library(ggplot2)
library(tidyr)

# Reshape the data from a wide format to a long format
data_melted_dataframe_1 <- dataframe_1 %>%
  pivot_longer(cols = -`Fraud Categories`,
               names_to = "Year",
               values_to = "Fraud Cases")

# View the reshaped data
print(data_melted_dataframe_1)
```

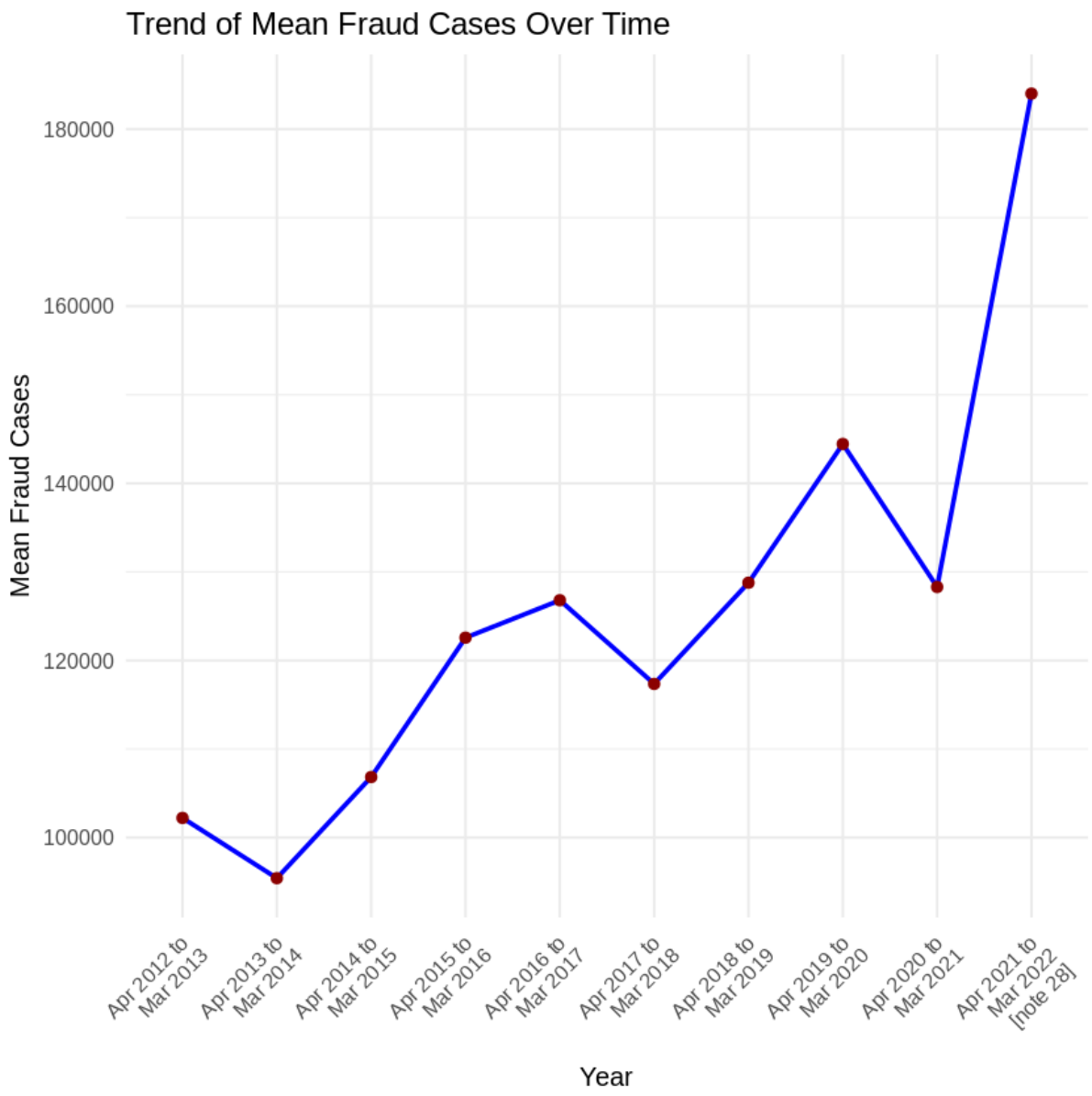
```
# A tibble: 60 × 3
   `Fraud Categories`                Year                          `Fraud Cases`
   <chr>                             <chr>                                 <dbl>
 1 Banking and credit industry fraud "Apr 2012 to \r\nMar 2013"            306641
 2 Banking and credit industry fraud "Apr 2013 to \r\nMar 2014"            286234
 3 Banking and credit industry fraud "Apr 2014 to \r\nMar 2015"            320495
 4 Banking and credit industry fraud "Apr 2015 to \r\nMar 2016"            367714
 5 Banking and credit industry fraud "Apr 2016 to \r\nMar 2017"            380390
 6 Banking and credit industry fraud "Apr 2017 to \r\nMar 2018"            352064
 7 Banking and credit industry fraud "Apr 2018 to \r\nMar 2019"            386326
 8 Banking and credit industry fraud "Apr 2019 to \r\nMar 2020"            433336
 9 Banking and credit industry fraud "Apr 2020 to \r\nMar 2021"            384886
10 Banking and credit industry fraud "Apr 2021 to \r\nMar 2022\r\…         552024
# i 50 more rows
```

```
# Line plot with Year on x-axis and Fraud Cases on y-axis
ggplot(data_melted_dataframe_1, aes(x = Year, y = `Fraud Cases`, group
= `Fraud Categories`, color = `Fraud Categories`)) +
  geom_line(size = 1) +
  geom_point(size = 2) +
  theme_minimal(base_size = 2) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1, size = 8),
        axis.text.y = element_text(size=10),
        plot.title = element_text(size = 18, face = "bold"),
        axis.title.x = element_text(size = 14),
        axis.title.y = element_text(size = 14),
        legend.title = element_text(size = 12),
        legend.text = element_text(size = 8)
        ) +
  scale_y_continuous(
    breaks = seq(0, 600000, by = 50000)) +
  labs(title = "Fraud Trends by Category Over Time", x = "Year", y =
"Fraud Cases", color = "Fraud Categories")
```

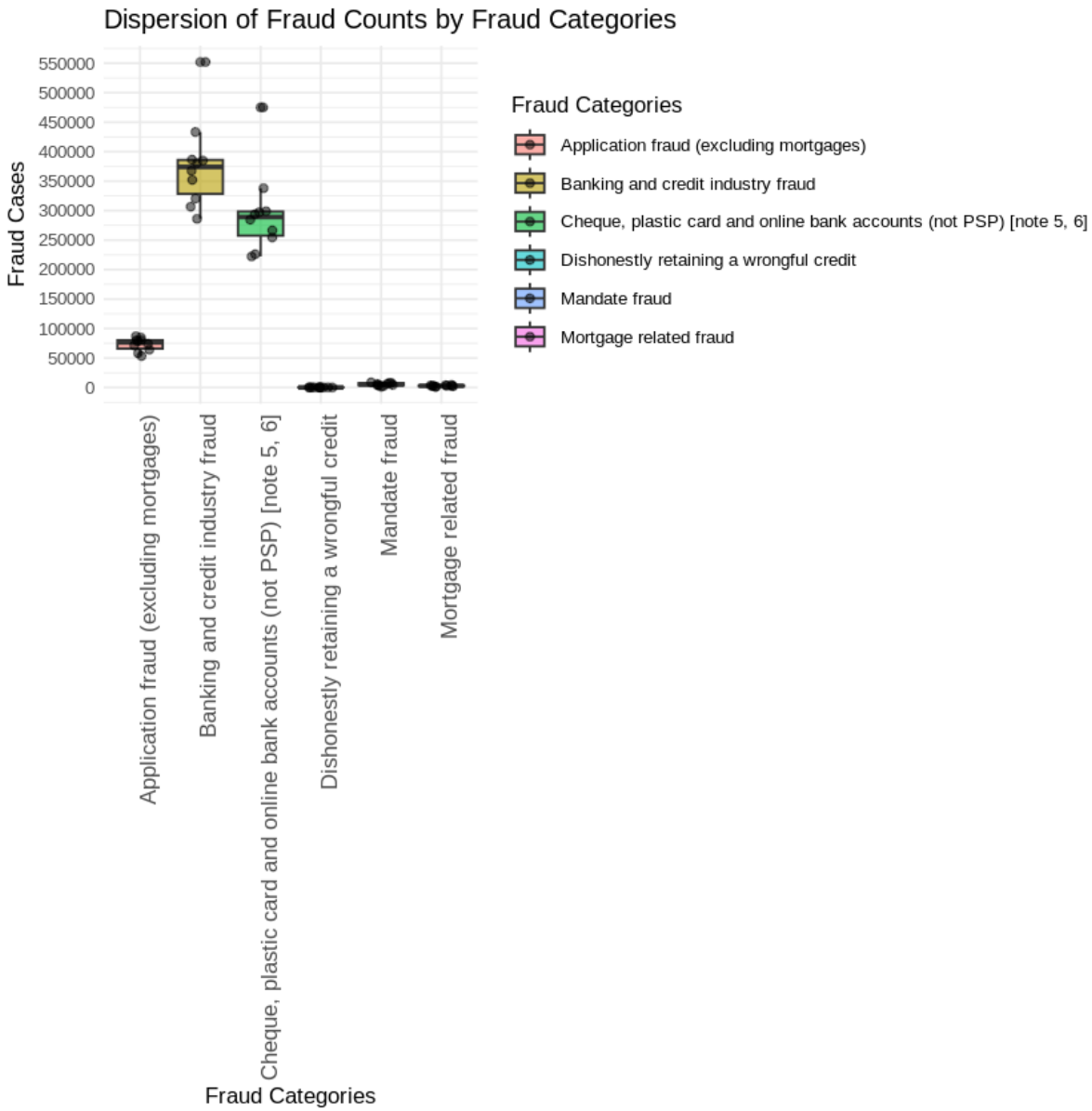



```r
# Calculating the mean for each year for all fraud categories
data_melted_summary <- data_melted_dataframe_1 %>%
  group_by(Year) %>%
  summarise(Mean = mean(`Fraud Cases`, na.rm = TRUE))

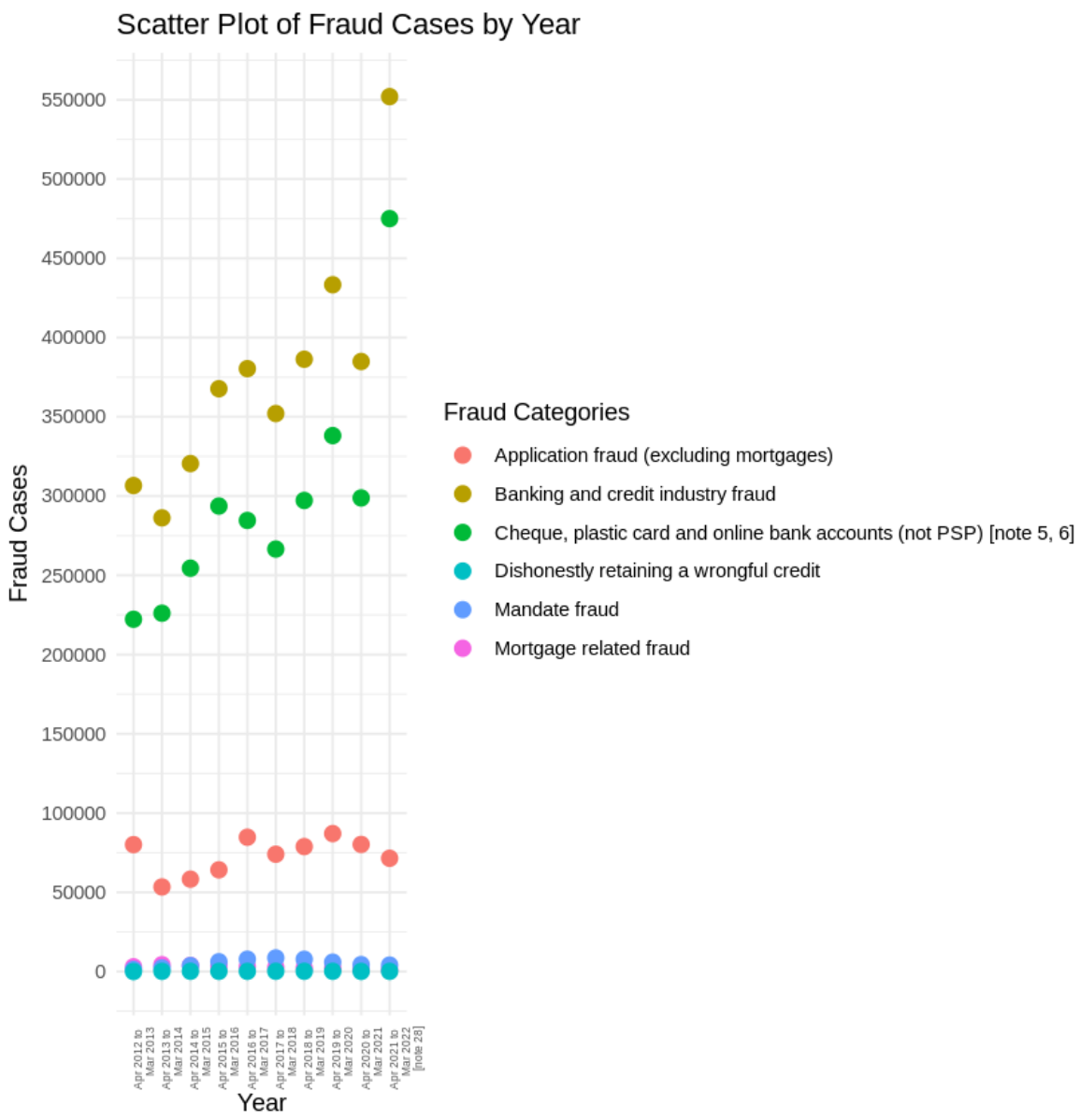
# Line chart to show central tendency mean over the years
ggplot(data_melted_summary, aes(x = Year, y = Mean, group = 1)) +
  geom_line(color = "blue", size = 1) +
  geom_point(color = "darkred", size = 2) +
  ggtitle("Trend of Mean Fraud Cases Over Time") +
  xlab("Year") +
  ylab("Mean Fraud Cases") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1))
```



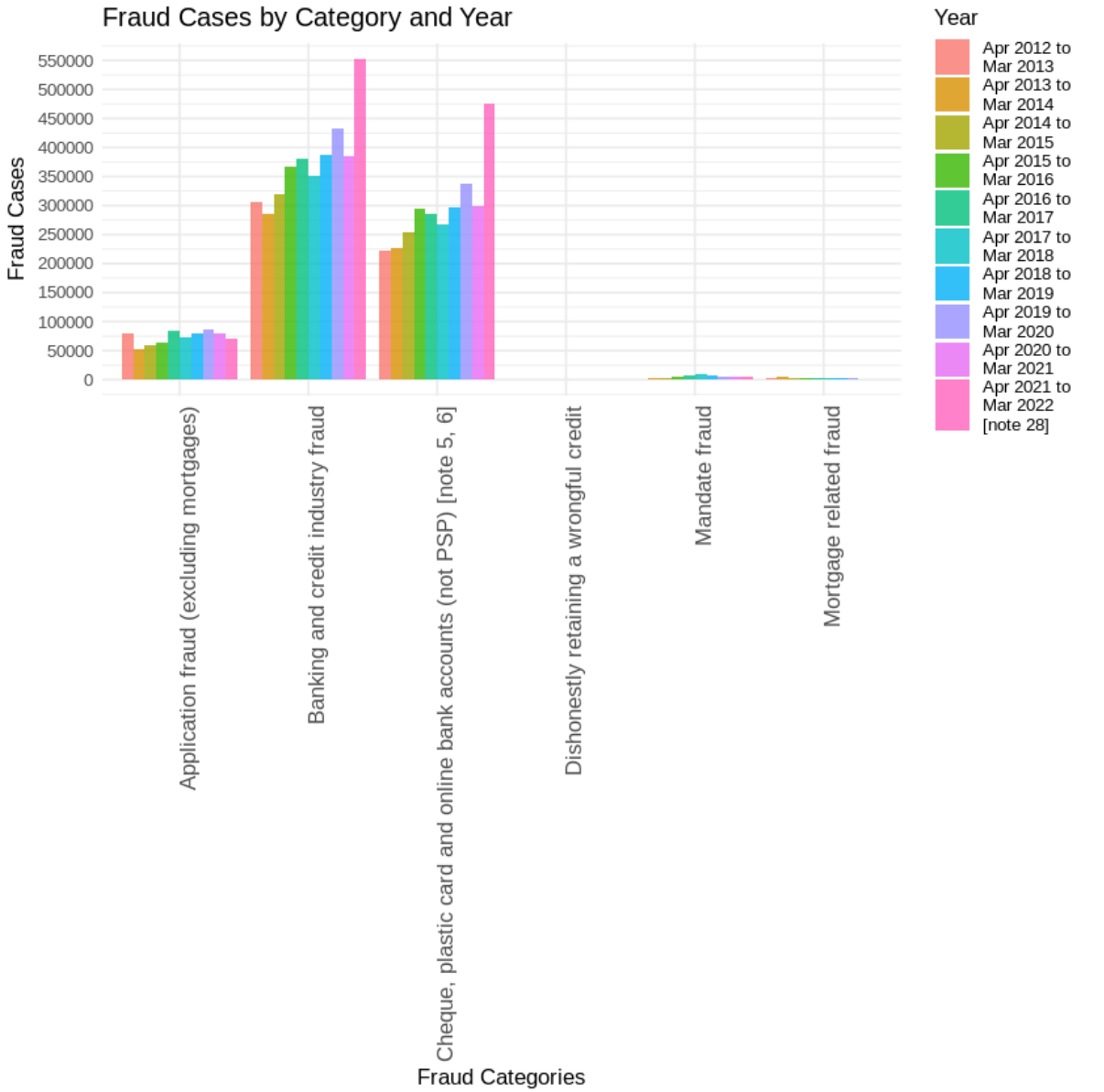

```
# Boxplot to visualise the dispersion across categories per year
ggplot(data_melted_dataframe_1, aes(x = `Fraud Categories`, y = `Fraud
Cases`, fill = `Fraud Categories`)) +
  geom_boxplot(alpha = 0.6) +
  geom_jitter(width = 0.2, alpha = 0.5) +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1, size = 10)) +
  scale_y_continuous(
    breaks = seq(0, 600000, by = 50000)) +
  labs(title = "Dispersion of Fraud Counts by Fraud Categories", x =
"Fraud Categories", y = "Fraud Cases")
```




```
# visualise the scatter plot for fraud cases by year
ggplot(data_melted_dataframe_1, aes(x = Year, y = `Fraud Cases`, color
= `Fraud Categories`)) +
  geom_point(size = 3, alpha = 1) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1, size = 5)
  ) +
    scale_y_continuous(
    breaks = seq(0, 600000, by = 50000)) +
  labs(
    title = "Scatter Plot of Fraud Cases by Year",
    x = "Year",
    y = "Fraud Cases",
    color = "Fraud Categories"
  )
```




```
# visualise a bar plot
ggplot(data_melted_dataframe_1, aes(x = `Fraud Categories`, y = `Fraud
Cases`, fill = Year)) +
  geom_bar(stat = "identity", position = "dodge", alpha = 0.8) +
  theme_minimal(base_size = 10) +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1, size = 10),
  ) +
  scale_y_continuous(
    breaks = seq(0, 600000, by = 50000)) +
  labs(
    title = "Fraud Cases by Category and Year",
    x = "Fraud Categories",
    y = "Fraud Cases",
    fill = "Year"
  )
```




## Code (Date Frame 2 Task 2) And Output

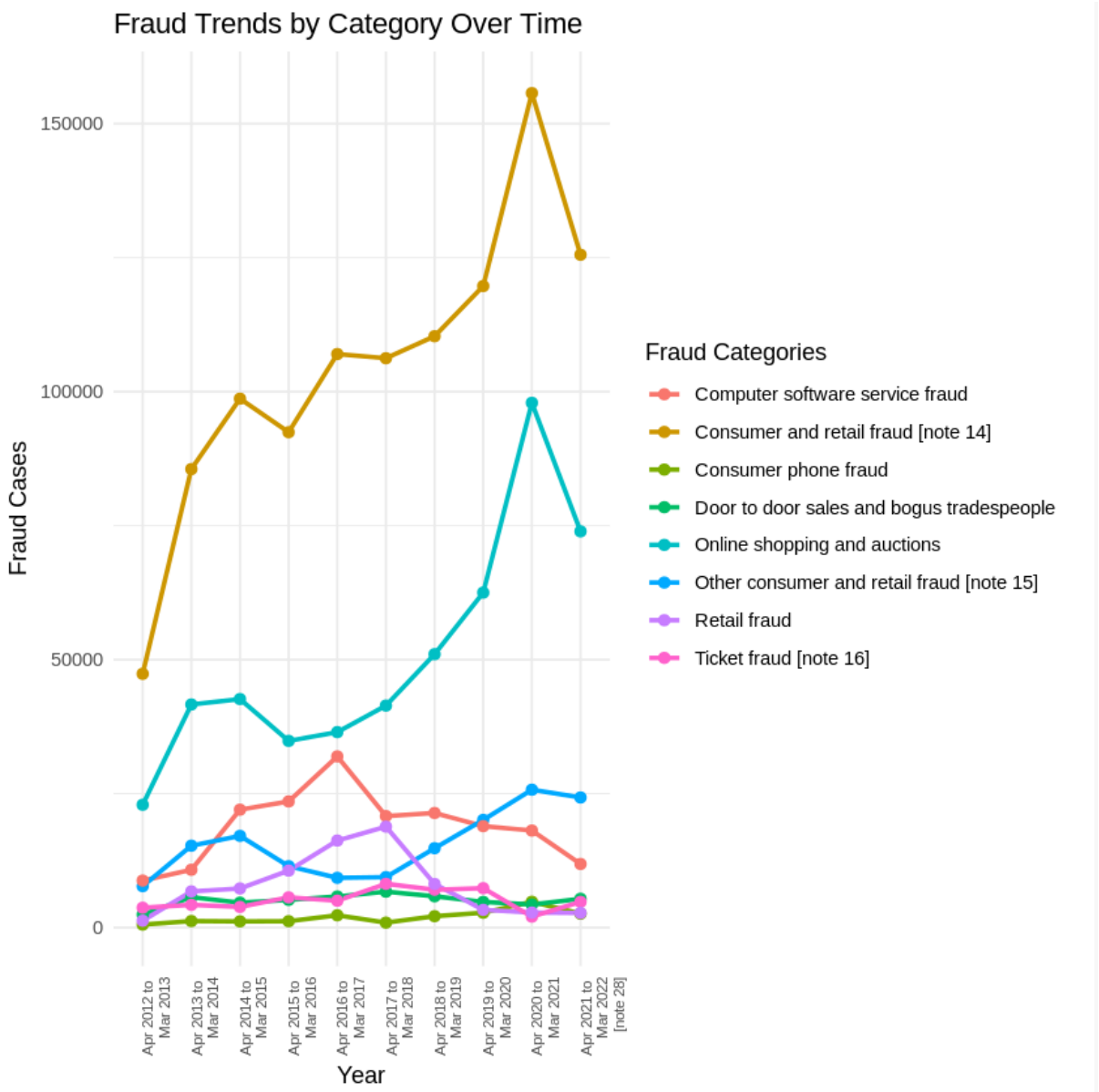
```
library(ggplot2)
library(tidyr)

# Reshape the data from wide to long format
data_melted_dataframe_2 <- dataframe_2 %>%
  pivot_longer(cols = -`Fraud Categories`,
               names_to = "Year",
               values_to = "Fraud Cases")

# View the reshaped data
print(data_melted_dataframe_2)
```

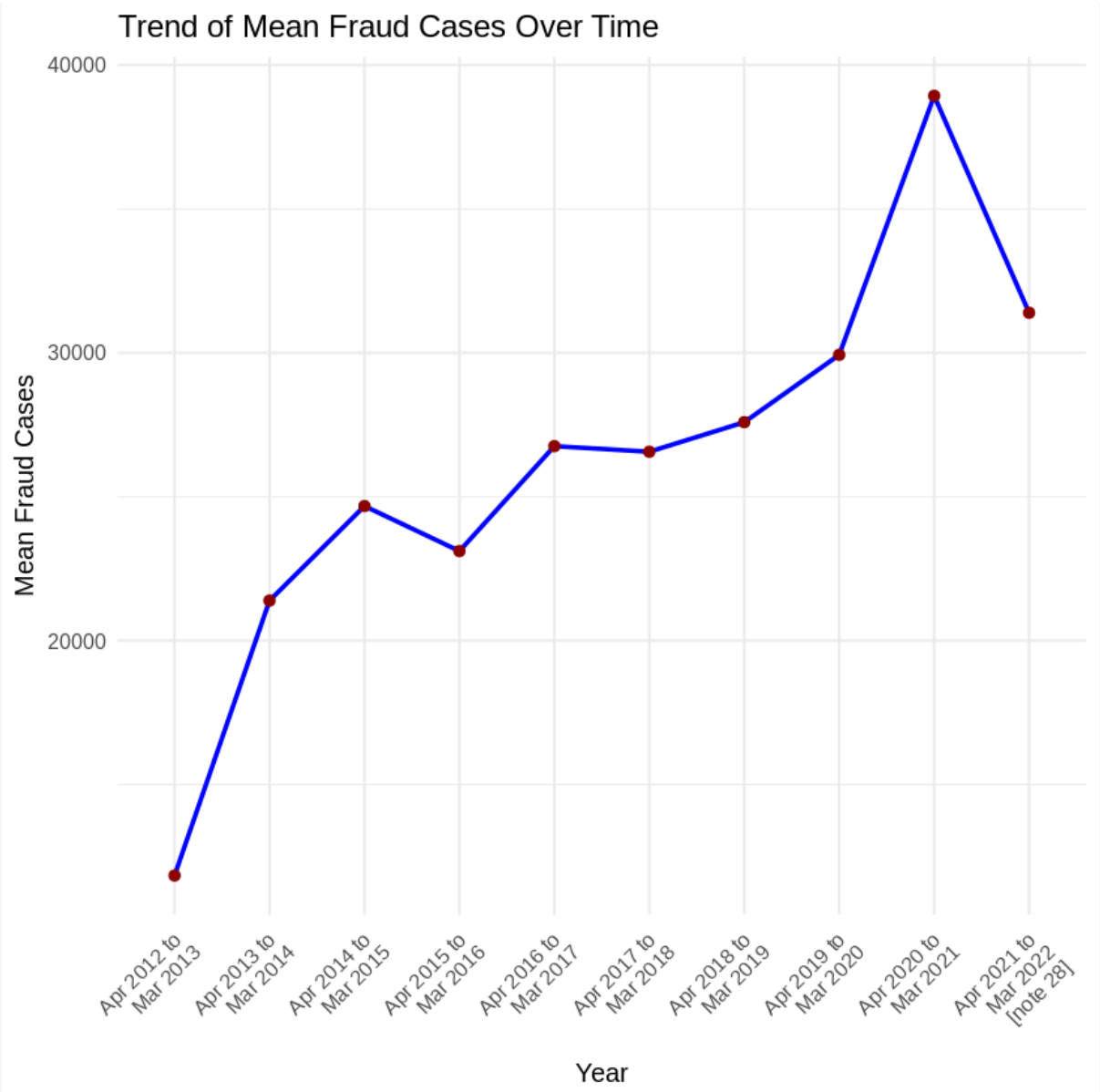
```
# A tibble: 80 × 3
   `Fraud Categories`                  Year                          `Fraud Cases`
   <chr>                               <chr>                                 <dbl>
 1 Consumer and retail fraud [note 14] "Apr 2012 to \r\nMar 2013"            47352
 2 Consumer and retail fraud [note 14] "Apr 2013 to \r\nMar 2014"            85560
 3 Consumer and retail fraud [note 14] "Apr 2014 to \r\nMar 2015"            98696
 4 Consumer and retail fraud [note 14] "Apr 2015 to \r\nMar 2016"            92442
 5 Consumer and retail fraud [note 14] "Apr 2016 to \r\nMar 2017"           107023
 6 Consumer and retail fraud [note 14] "Apr 2017 to \r\nMar 2018"           106249
 7 Consumer and retail fraud [note 14] "Apr 2018 to \r\nMar 2019"           110355
 8 Consumer and retail fraud [note 14] "Apr 2019 to \r\nMar 2020"           119716
 9 Consumer and retail fraud [note 14] "Apr 2020 to \r\nMar 2021"           155717
10 Consumer and retail fraud [note 14] "Apr 2021 to \r\nMar 2022\…           125560
# ℹ 70 more rows
```

```r
# Line plot with Year on x-axis and Fraud Cases on y-axis
ggplot(data_melted_dataframe_2, aes(x = Year, y = `Fraud Cases`, group
= `Fraud Categories`, color = `Fraud Categories`)) +
  geom_line(size = 1) +
  geom_point(size = 2) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1, size = 7)) +
  scale_y_continuous(
    breaks = seq(0, 600000, by = 50000)) +
  labs(title = "Fraud Trends by Category Over Time", x = "Year", y =
"Fraud Cases", color = "Fraud Categories")
```

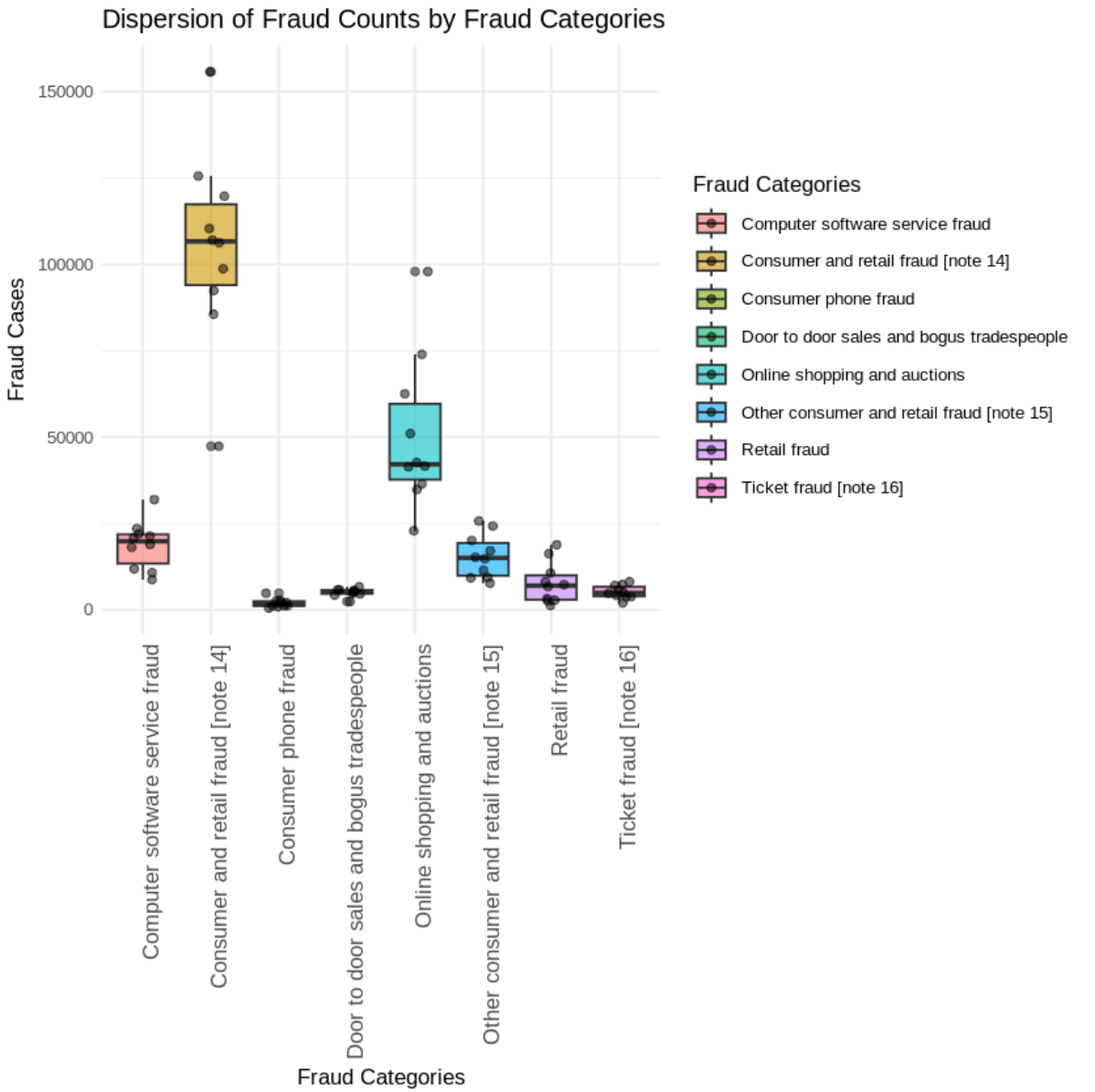



```r
# calculate mean for each year against all fraud categories
data_melted_summary <- data_melted_dataframe_2 %>%
  group_by(Year) %>%
  summarise(Mean = mean(`Fraud Cases`, na.rm = TRUE))

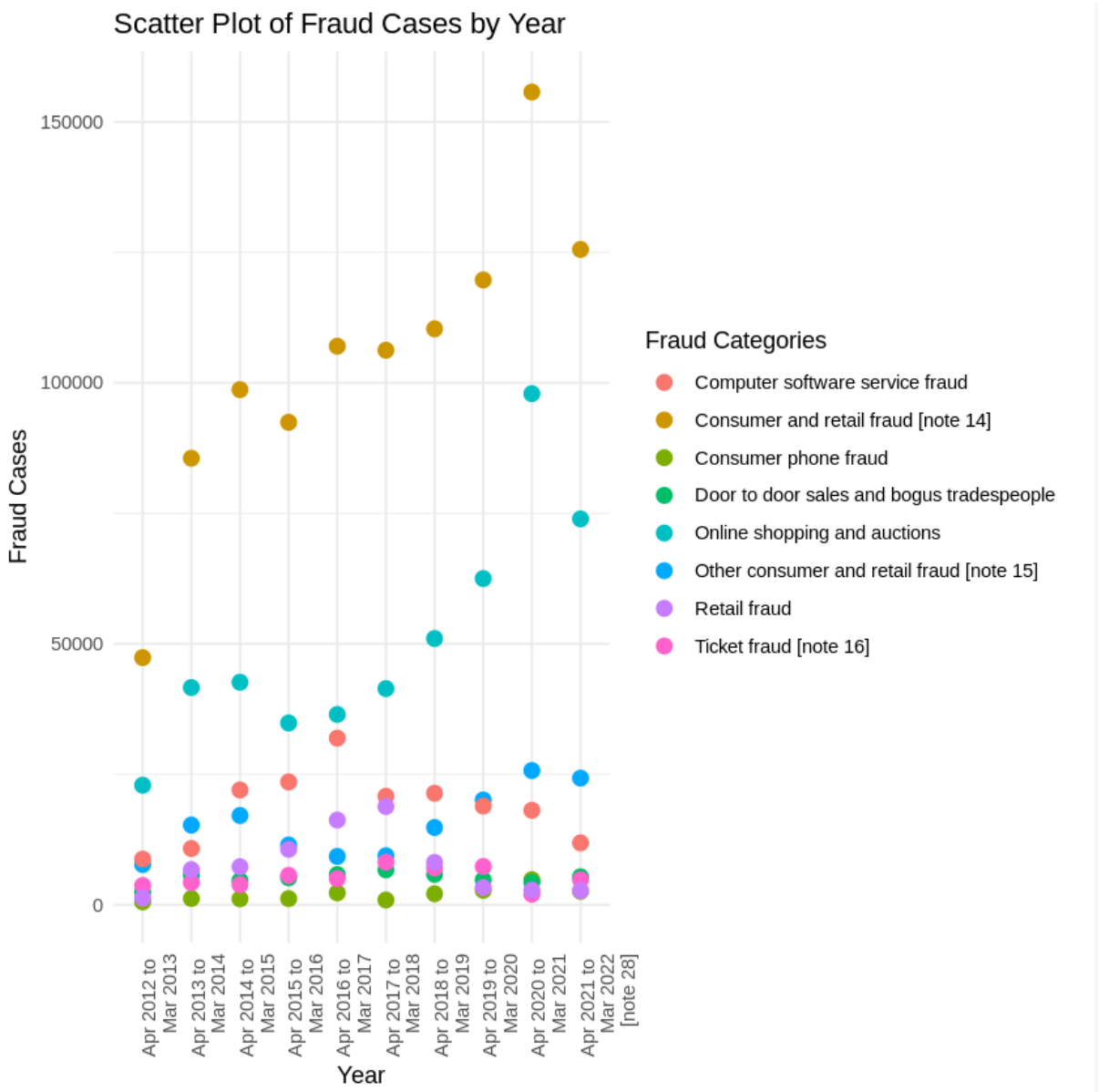
# visualise a line chart
ggplot(data_melted_summary, aes(x = Year, y = Mean, group = 1)) +
  geom_line(color = "blue", size = 1) +
  geom_point(color = "darkred", size = 2) +
  ggtitle("Trend of Mean Fraud Cases Over Time") +
  xlab("Year") +
  ylab("Mean Fraud Cases") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1))
```



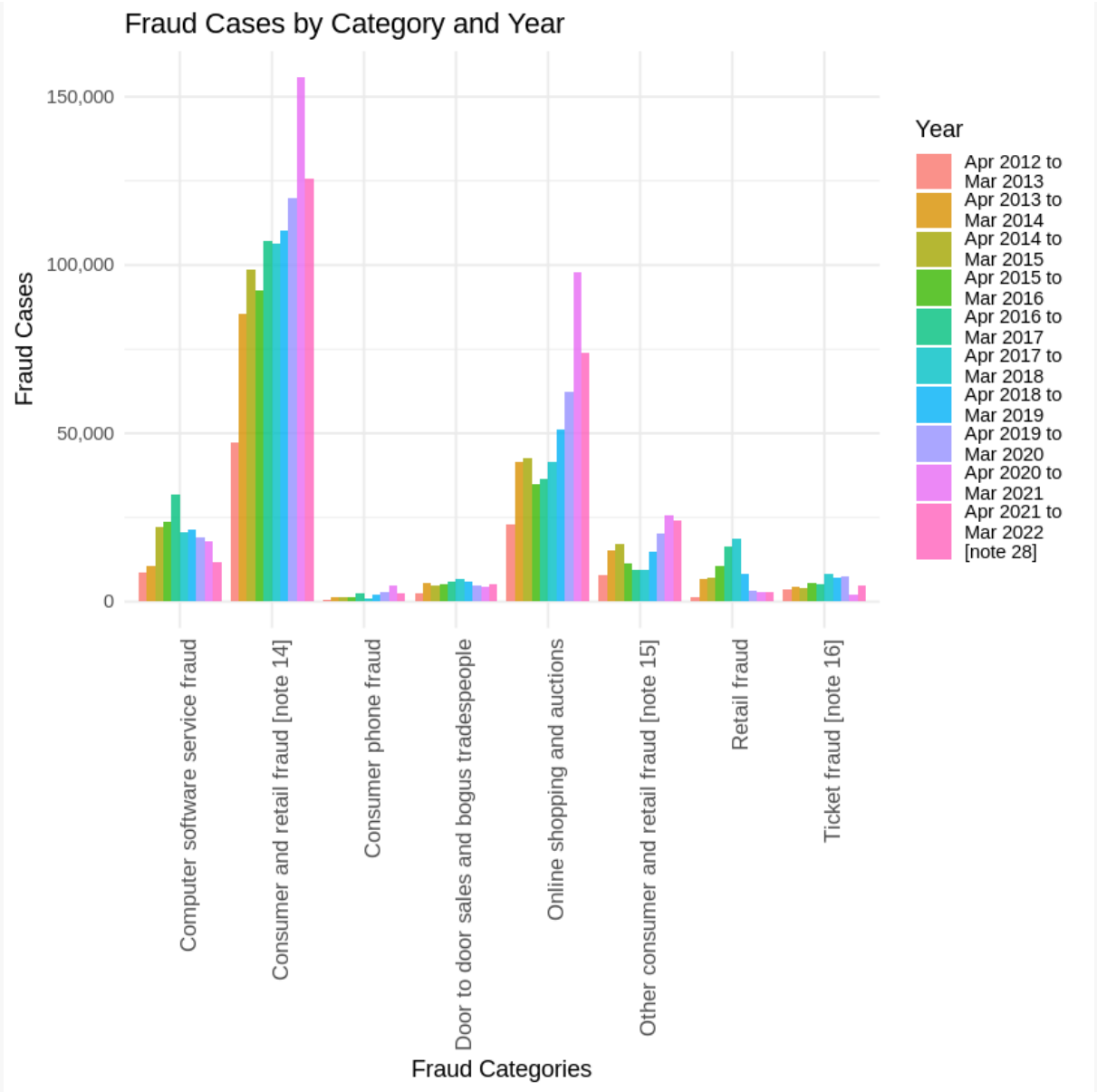

```
# visualise a boxplot for dispersion across fraud categories per year
ggplot(data_melted_dataframe_2, aes(x = `Fraud Categories`, y = `Fraud
Cases`, fill = `Fraud Categories`)) +
  geom_boxplot(alpha = 0.6) +
  geom_jitter(width = 0.2, alpha = 0.5) +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1, size = 10)) +
  scale_y_continuous(
    breaks = seq(0, 600000, by = 50000)) +
  labs(title = "Dispersion of Fraud Counts by Fraud Categories", x =
"Fraud Categories", y = "Fraud Cases")
```




```
# visualise a scatter plot for fraud cases by year
ggplot(data_melted_dataframe_2, aes(x = Year, y = `Fraud Cases`, color
= `Fraud Categories`)) +
  geom_point(size = 3, alpha = 1) + # add scatter points
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1)
  ) +
  labs(
    title = "Scatter Plot of Fraud Cases by Year",
    x = "Year",
    y = "Fraud Cases",
    color = "Fraud Categories"
  )
```




```
# visualise a bar chart
ggplot(data_melted_dataframe_2, aes(x = `Fraud Categories`, y = `Fraud
Cases`, fill = Year)) +
  geom_bar(stat = "identity", position = "dodge", alpha = 0.8) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1, size = 10),
  ) +
  scale_y_continuous(labels = scales::comma_format()) +
  labs(
    title = "Fraud Cases by Category and Year",
    x = "Fraud Categories",
    y = "Fraud Cases",
    fill = "Year"
  )
```


Fraud Cases by Category and Year
Fraud Cases
150,000
100,000
50,000
0
Computer software service fraud
Consumer and retail fraud [note 14]
Consumer phone fraud
Door to door sales and bogus tradespeople
Online shopping and auctions
Other consumer and retail fraud [note 15]
Retail fraud
Ticket fraud [note 16]
Fraud Categories
Year
Apr 2012 to Mar 2013
Apr 2013 to Mar 2014
Apr 2014 to Mar 2015
Apr 2015 to Mar 2016
Apr 2016 to Mar 2017
Apr 2017 to Mar 2018
Apr 2018 to Mar 2019
Apr 2019 to Mar 2020
Apr 2020 to Mar 2021
Apr 2021 to Mar 2022 [note 28]